CS336 · Spring 2026 · Lecture 08

并行训练基础：从集合通信到大模型训练布局

如何阅读这份自学增强讲义

原始英文幻灯片完整保留；每组原页之后紧接对应中文讲解。书签可跳转到原页与讲解。页码中的“原页”始终指原始 PDF 页码。

中文部分为自学补充；课堂转述、原论文结论与编者推导应按文中标记区分。第三方保存的字幕仅用于检索定位，不等同于教师审校逐字稿。

学习目标：读完后应能画出 AllReduce、ReduceScatter、AllGather 的数据去向；推导 DDP 与 ZeRO 各阶段的每卡常驻状态和通信量；用矩阵分块说明 TP 的前反向通信；计算 PP 的气泡；把 SP、CP、EP 放到正确的张量轴与通信组中，并解释真实训练为什么组合多种并行。

先修：矩阵乘法及链式法则、SGD/Adam、Transformer 的注意力和 MLP、GPU 显存与带宽。阅读主线是“哪些状态留在每张卡→谁计算哪部分→何时交换什么→峰值显存与关键路径”。先读原页，再读其后讲解；所有手算与调度推导标为编者补充，不冒充教师原话。

统一记号：P 为参数个数；d 为数据并行组大小；t 为张量并行组大小；p 为流水线阶段数；m 为一次更新中的微批数；b 为每个数据并行副本的微批样本数；s 为序列长度；h 为隐藏维度；a 为注意力头数；q 为一次通信中每元素字节数。独立的普通三维并行满足 $N=d\times t\times p$；全局批量 $B=d\times m\times b$。EP 常复用已有轴，不能不加判断地再乘 EP。

通信默认统计每个 rank（进程/设备）发送的载荷，不把发送与接收相加，不把各卡再相加；环算法中接收量相同。显存默认以十进制 GB 计，容量表仅为模型状态预算，不包含激活、预取、通信工作区与碎片。算法的常数必须连同精度、调度和分组前提阅读。

来源索引：课程官网确认 Spring 2026，第8讲原 PDF 是本稿结构基准；缓存字幕只用于核对课堂补充时间段。NCCL 文档核对集合操作，ZeRO/FSDP 原文核对状态分片，Megatron 与 activation 论文核对调度及显存，其他资料服务于对应扩展主题。原页中的历史硬件数据和示例配置按课堂案例阅读，不作为当前所有系统的统一建议。

Lecture 8

PARALLELISM BASICS

CS336

Tatsu H

Outline and goals

- Understand the systems complexities of training huge models
- Different parallelization paradigms and why people use multiple at once
- What large scale training runs often look like

Organization today:

- **Part 1**: Basics of networking for LLMs
- **Part 2**: Different forms of parallel LLM training
- **Part 3**: Scaling and training big LMs with parallelism

第 8 讲 · 原页 1–3 · 中文讲解

本讲的问题：怎样把一个训练任务放到一群设备上

三条线索

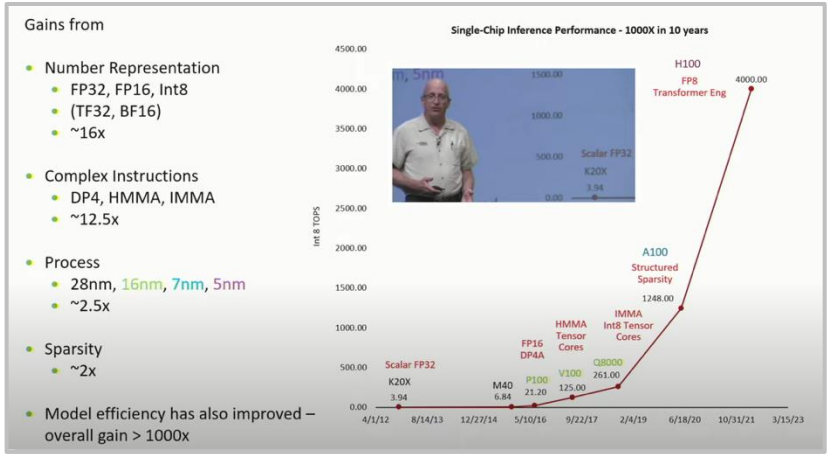
网络部分回答“交换一次张量有多贵”；并行范式回答“训练状态和计算归谁”；案例部分回答“怎样让布局与机器、批量和上下文匹配”。这些问题互相约束：能装进显存不等于吞吐高，FLOPs 分得均匀也不等于通信能被隐藏。读案例时先辨认瓶颈，再看并行度，而不要先记 TP=8 这样的数字。

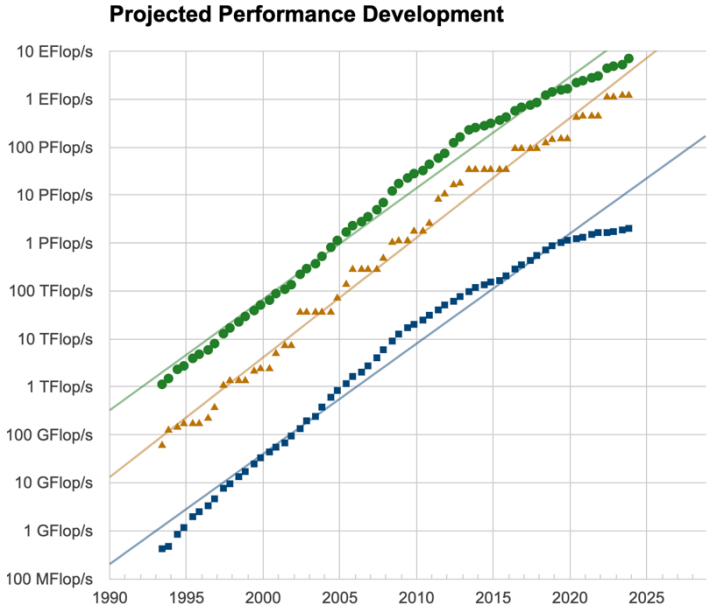
训练一步的依赖链

输入→各层前向→损失→各层反向→梯度同步→优化器更新。前向激活要保留或重算，反向既求输入梯度又求权重梯度，Adam 还需历史矩状态。并行可以改动这些对象的存储位置、执行位置和时间顺序，但同步训练必须让一次更新对应同一全局批量、同一版本的权重。

本节参考（完整书目见讲义末尾）：[slides]

Limits to GPU-based scaling – compute



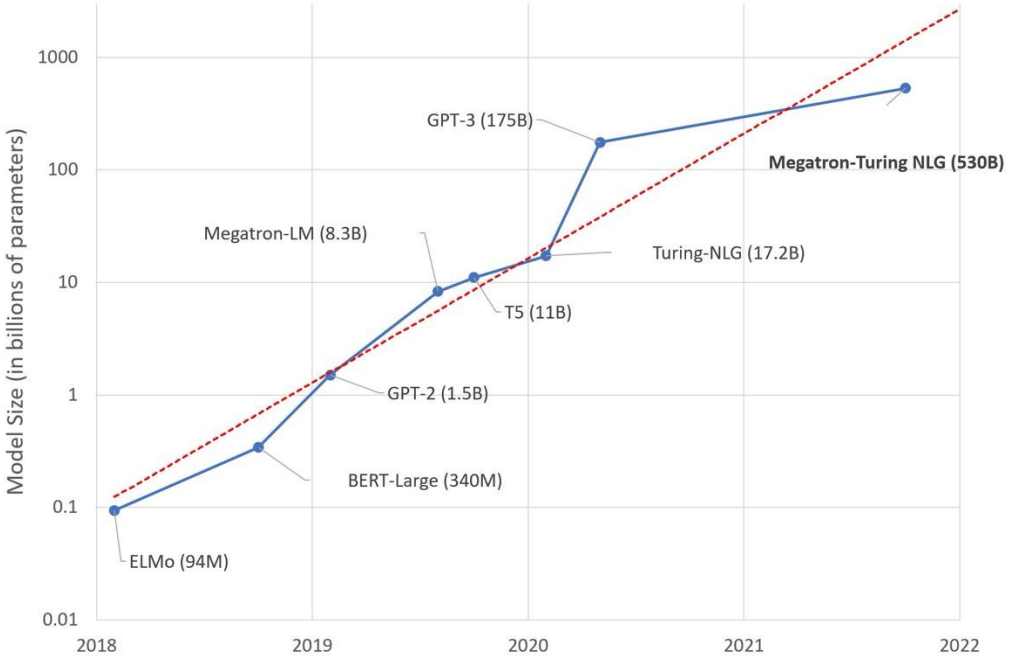


There are limits to single-GPU scaling.

The word's fastest supercomputers have *exaflops* of compute

Limits to GPU-based scaling - memory

Models are getting really big..

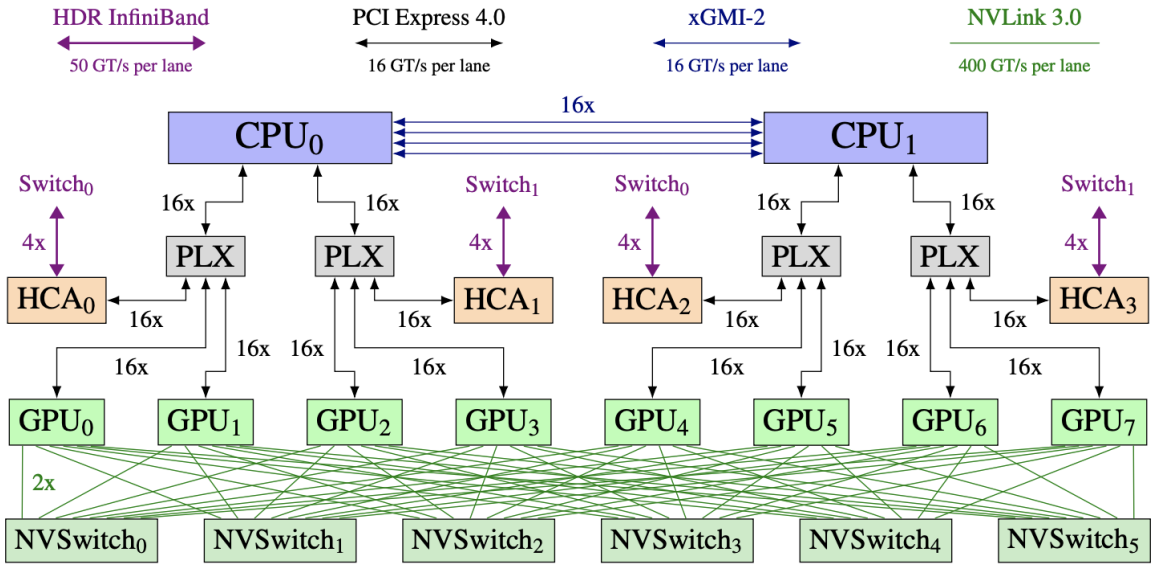


A single GPU can’t fit most of these large models!

What do we do? Multi-GPU, multi-machine parallelism

Intra-node parallelism
via high-speed interconnects

High-speed inter-node parallelism



Split up memory and compute requirements across GPUs and machines

第 8 讲 · 原页 4–6 · 中文讲解

为什么单卡不够：算力、容量和互联是三份预算

读图与动机

第4页展示计算能力的历史增长，第5页展示参数量增长；两张趋势图只能说明需求扩张，不能直接拿不同精度、不同年代的峰值互相比有效训练速度。粗略训练 FLOPs 可以很大，而单卡容量是另一道硬约束：即使愿意等很久，一个必须同时驻留的张量超出显存也无法执行。

编者手算：权重只是起点

P=70×10^9 的模型，仅 BF16 参数就是 2P=140 GB；若使用本讲第17页的16字节混合精度 Adam 状态，则是1120 GB，尚未包括激活。8张80 GB卡合计640 GB，并不意味着DDP能训练它：DDP把状态复制八份，每张卡仍需1120 GB。只有分片或卸载才可能使用合计容量。

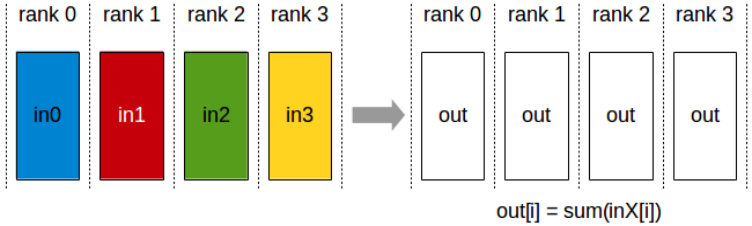
第6页互联图怎么看

GPU 之间的高速直连/交换网络承担机内通信；跨节点通常经过网卡与交换机，路径、带宽和延迟不同。数据搬运不会因为拥有更多FLOPs自动加速。一个有用的粗模型是 T≈计算时间+未被计算覆盖的通信时间+等待；仅在理想重叠且无资源争用时，某段时间才能近似 max(T计算,T通信)。

本节参考（完整书目见讲义末尾）：[slides]

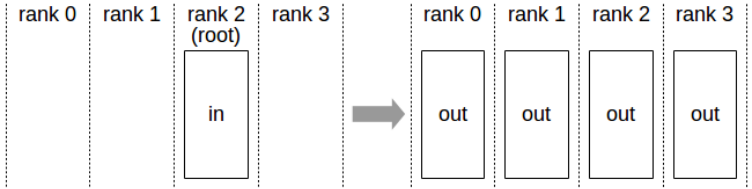
But first.. Some basics about collective communication

All reduce



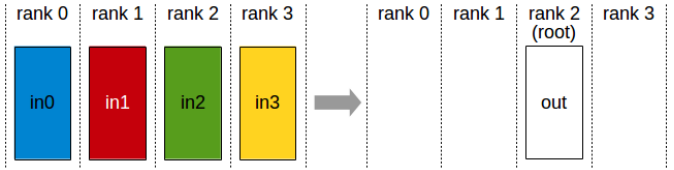
out[i] = sum(inX[i])

Broadcast



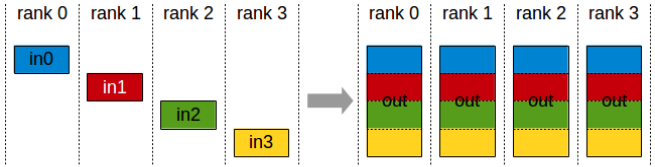
out[i] = in[i]

Reduce



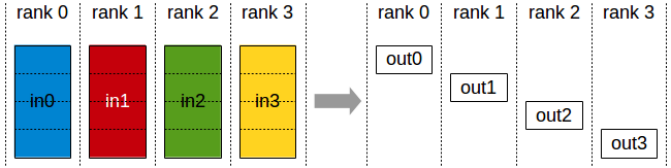
out[i] = sum(inX[i])

All Gather



out[Y*count+i] = inY[i]

Reduce Scatter



outY[i] = sum(inX[Y*count+i])

第 8 讲 · 原页 7–7 · 中文讲解

五种集合操作：先弄清结果放在哪里

从两张卡手算语义

设rank0有向量[1,2,3,4]，rank1有[10,20,30,40]。求和Reduce把[11,22,33,44]只交给指定root；AllReduce让两卡都得到这个完整向量。Broadcast从root复制它已有的数据，不做求和。AllGather把不同卡持有的片段按rank顺序拼接，例如[11,22]与[33,44]变成每卡都有[11,22,33,44]。它不是求平均。

ReduceScatter与训练的联系

ReduceScatter先对同位置元素求和，再分片交付：上述输入使rank0只得到[11,22]，rank1只得到[33,44]。所以“每卡仅保留一部分”不代表丢掉了其他卡的贡献。对于梯度，我们可让每个参数片段的负责卡得到全体样本的梯度，再更新相应权重。第7页彩色块表示不同rank贡献，输出的位置和长度比颜色本身更重要。

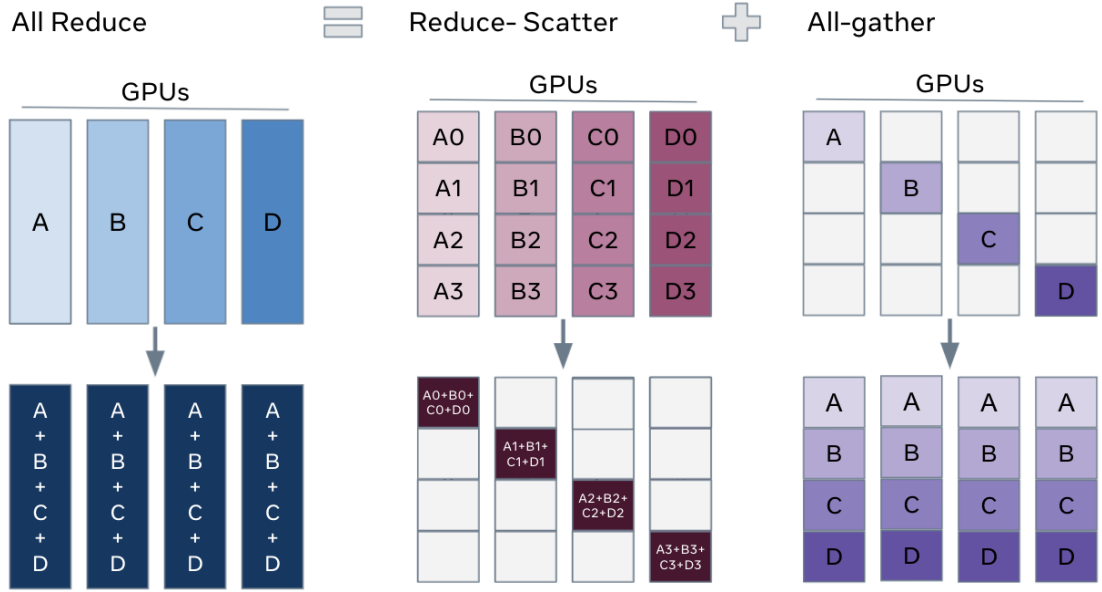
调用契约

参与同一个集合操作的rank必须一致地执行匹配的调用，形状、元素类型、归约运算和顺序需要满足接口要求；不能一张卡因分支跳过而让其他卡永远等待。sum与mean是不同语义：若接口做sum，平均梯度的除数应由训练逻辑处理。本文用实数加法解释，浮点归约顺序变化可能带来舍入差异。

本节参考（完整书目见讲义末尾）：[slides] · [nccl]

Important detail – all reduce vs reduce-scatter-gather.

Reduce can be implemented as two steps: reduce-scatter and all-gather



Importantly, in the bandwidth-limited regime, this is the best you can do

第 8 讲 · 原页 8–8 · 中文讲解

AllReduce的分解与“2P通信量”的来源

原稿措辞澄清

本页图示正确表达的是 AllReduce = ReduceScatter + AllGather；正文写“Reduce can be implemented…”省略了All。单root的Reduce本身并不要求把结果重新广播给所有rank。把两步拆开后，ZeRO就能在“得到梯度片段”和“汇集结果”之间插入本地优化器更新。

编者推导：环算法的字节预算

设组内n张卡，每张输入长度为L个元素，每元素q字节，S=qL。把输入分成n块。ReduceScatter有n−1轮，每轮每卡发送S/n字节，于是发送量为(n−1)S/n；AllGather同样发送(n−1)S/n。总计V=2(n−1)S/n，n很大时近似2S。n=4、S=8 MB时，每阶段6 MB，总发送12 MB；每卡另接收12 MB。如果统计整个组的发送量，则要再乘4。

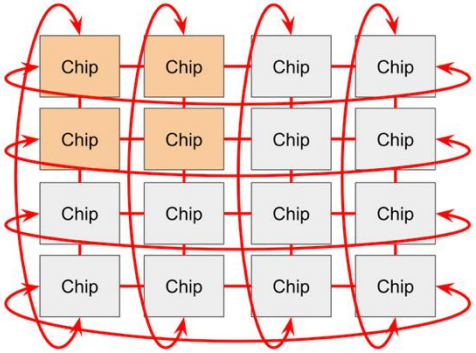
时间与最优性的前提

用α表示每轮启动延迟，β表示有效链路带宽，理想环AllReduce时间约2(n−1)α+2(n−1)S/(nβ)。这是均匀分块、网络不拥塞、无跨链路额外瓶颈的估算。在大消息、带宽主导时这种分解具有良好效率；小消息、树形算法、硬件内归约、分层网络会改变轮数和瓶颈。“最佳”不是对任何拓扑和消息大小都成立。

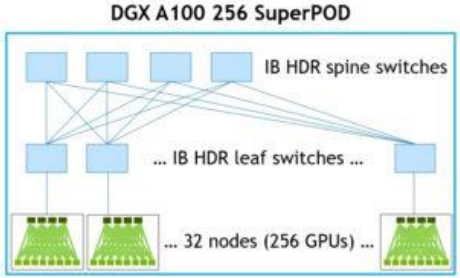
本节参考（完整书目见讲义末尾）：[slides] · [nccl] · [zero]

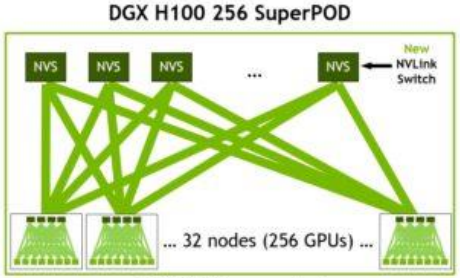
TPUs vs GPUs – design differences at the comm level

TPU networking
toroidal mesh



GPU networking
All-to-all (up to 256)





Fully NVLink-connected
Massive bisection bandwidth

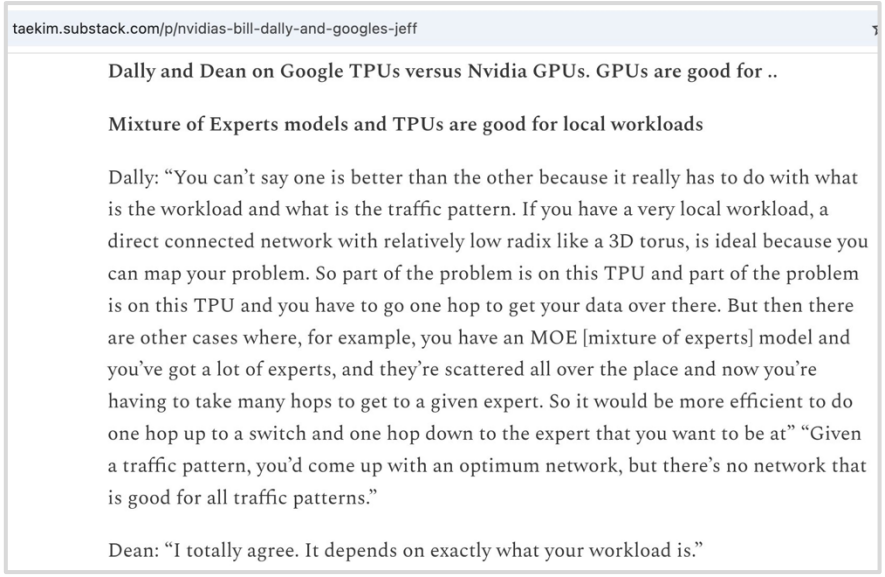
	A100 SuperPod			H100 SuperPod			Speedup	
	Dense PFLOP/s	Bisection [GB/s]	Reduce [GB/s]	Dense PFLOP/s	Bisection [GB/s]	Reduce [GB/s]	Bisection	Reduce
1 DGX / 8 GPUs	2.5	2,400	150	16	3,600	450	**1.5x**	**3x**
32 DGXs / 256 GPUs	80	6,400	100	512	57,600	450	**9x**	**4.5x**

Mesh vs tree vs other

Why mesh? Why tree?

Pro mesh – Cheaper to operate, can be made fast (and just do tensor parallel)

Pro tree/A2A – Better for less structured communications (expert parallel)



第 8 讲 · 原页 9–10 · 中文讲解

拓扑决定通信代价：环面、树与交换网络

图中的连线意味着什么

第9页TPU示意图是带回绕边的网格/环面：每个节点只直接连少量邻居，远距离传输需多跳。GPU示例利用交换结构获得较灵活的设备间连接；“all-to-all”在这里描述互通能力，不保证每一对GPU都有独立物理线，也不等于运行了一次AllToAll集合操作。

为什么不同工作负载偏好不同网络

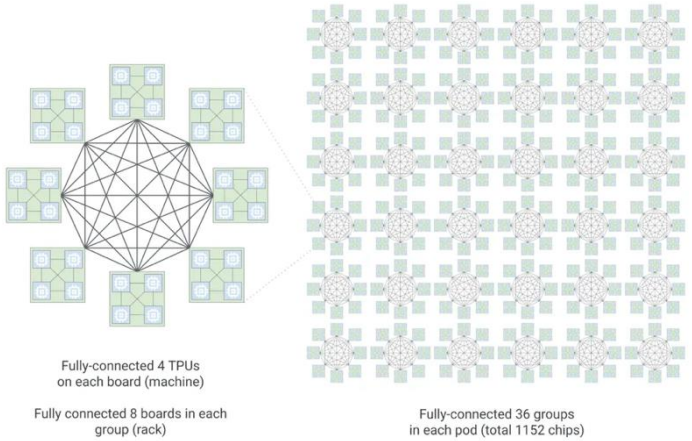
规则的矩阵切分和固定集合操作容易安排路由；MoE按token动态路由，目标设备不均匀，可能给少数链路造成热点。树/交换网络的优势取决于是否超额订阅、总交换容量及网络双分带宽，即把设备群分成两半时能跨过去多少数据。便宜的邻接连线与灵活的全局交换各有代价；不能只比较单端口带宽。

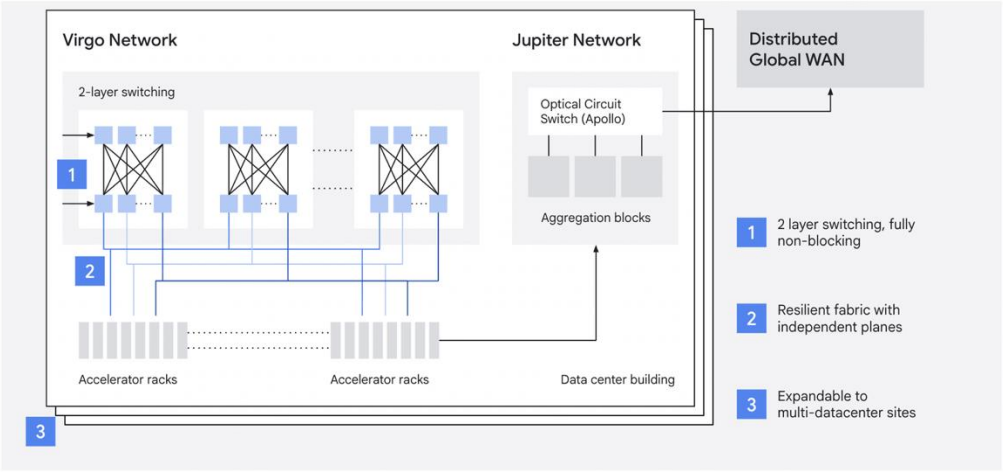
边界

原页给出特定代际硬件示例。第11页紧接着展示设计变化，正说明“TPU永远网格、GPU永远全连接”不是应背的分类。实际选择要看当前集群的高速域边界，而不是芯片品牌。

本节参考（完整书目见讲义末尾）：[slides]

But then things change.. TPU8i/t





TPU8i networking – closer to tree-style topologies (maybe for MoEs?)
TPU8t scale-out networking – ‘Virgo’ with switched networks

Why not connect everything?

Domain sizes – why not connect everything?

Key Capabilities - Huawei Ascend 910C Cloud Matrix 384 vs Nvidia GB200 NVL72				
Chip and Package Level				
	Unit	**GB200**	**Ascend 910C**	**Huawei vs Nvidia**
BF16 dense TFLOPS	TFLOPS	2,500	780	0.3x
HBM capacity	GB	192	128	0.7x
HBM bandwidth	TB/s	8.0	3.2	0.4x
Scale Up Bandwidth	Gb/s uni-di	7,200	2,800	0.4x
Scale Out Bandwidth	Gb/s uni-di	400	400	1.0x
System Level				
	Unit	**Nvidia GB200 NVL72**	**Cloud Matrix CM384**	**Huawei vs Nvidia**
BF16 dense PFLOPS	PFLOPS	180	300	1.7x
HBM capacity	TB	13.8	49.2	3.6x
HBM bandwidth	TB/s	576	1,229	2.1x
Scale Up Bandwidth	Gb/s uni-di	518,400	1,075,200	2.1x
Scale Up Domain Size	GPUs	72	384	5.3x
Scale Out Bandwidth	Gb/s uni-di	28,800	153,600	5.3x
All-In System Power1	W	145,000	599,821	4.1x
All-in Power per BF16 dense FLOP	W/TFLOP	0.81	2.00	2.5x
All-in Power per memory bandwidth	W per TB/s	251.7	488.1	1.9x
All-in Power per memory capacity	kW/TB	10.5	12.2	1.2x

1. All-in System Power is total cluster power including scale-out networking, storage, etc.



https://newsletter.semianalysis.com/p/huawei-ai-cloudmatrix-384-chinas-answer-to-nvidia-gb200-nvl72

第 8 讲 · 原页 11–12 · 中文讲解

高速域不能无限扩大：把拓扑当成本问题

按图理解而非外推规格

第11页把新一代TPU网络与前页对照，其中“maybe for MoEs”是课堂推测，不是已证明的设计动机。右图区分加速器机架、数据中心网络和广域网络，强调多级网络。第12页对比表与机柜照片展示大规模高速互联需要交换设备、布线和供电空间；此处保留原图，不把第三方分析表中的数字转写成官方规格。

编者补充：为什么不是所有卡都直接相连

N个端点若两两专线，链路数为N(N−1)/2；N=8时28条，N=64时2016条。实际系统用交换机共享路径，以端口、光模块、线缆、距离、功耗和故障复杂度换取规模。即便逻辑上任意卡可互通，也未必能让所有端点同时满速互传。将频繁的TP通信留在高速域，把较少的PP边界通信放到域间，是后文策略的物理原因。

本节参考（完整书目见讲义末尾）：[slides]

Part 1 recap

- New unit of compute – the datacenter
- What we want from multi-machine scaling:
  - Linear memory scaling (max model params scales with num gpus)
  - Linear compute scaling (model flops scale linearly with num gpus)
- Simple collective comms primitives

Part 2 – Standard LLM parallelization primitives

How do we parallelize LLMs? 3 important ideas

- **Data parallelism**
  - Naïve data parallel
  - ZeRO levels 1-3
- **Model parallelism**
  - Pipeline parallel
  - Tensor parallel
- **Activation parallelism**
  - Sequence parallel

第 8 讲 · 原页 13–14 · 中文讲解

从网络到并行：区分数据轴、参数轴和激活轴

两种线性扩展不是一回事

理想计算扩展指同一工作量在N卡上用约1/N时间；理想容量扩展指可以容纳的模型状态随N增长。DDP可能获得前者却没有后者；ZeRO-3改善状态容量，却未必在小批量时获得好的计算扩展。第13页是目标，不是已保证的结果。

阅读分类表的方法

DP让不同卡读不同样本，参数更新保持一致；ZeRO在DP组中改变状态的存放方式。PP按层深分工，TP按层内矩阵维度分工；两者都让多张卡协作处理同一个样本。SP把可按token独立处理的激活沿序列分开。随后EP按专家分工、CP把注意力上下文也分开。回答“分了什么”之后，还应回答“什么时候需要重新汇集”。

本节参考（完整书目见讲义末尾）：[slides]

Naïve data parallelism

Starting point – imagine we are doing naïve SGD

$$\theta_{t+1} = \theta_t - \eta \sum_{i=1}^{B} \nabla f(x_i)$$

Naive parallelism: split the elements of B sized batch across M machines. Exchange gradients to synchronize

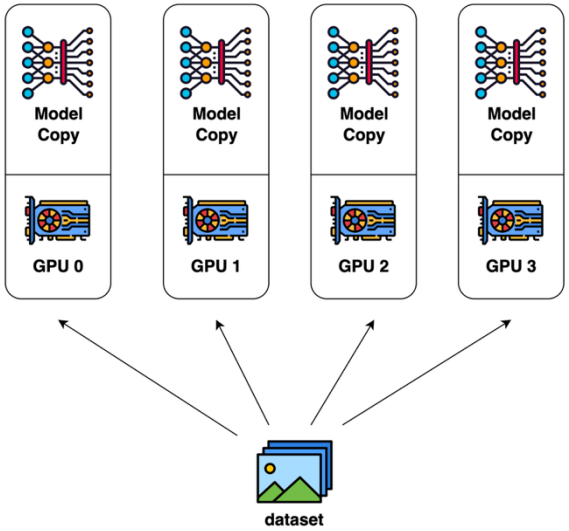
How does this do?

Compute scaling – each GPU gets B/M examples.

Communication overhead – transmits 2x # params every batch. OK if batches are big

Memory scaling – none. Every GPU needs # params at least

What's wrong with naïve data parallel?



Memory seems like it'd be a problem – we copy the model parameters to each GPU.

Let's take a closer look..

第 8 讲 · 原页 15–16 · 中文讲解

朴素数据并行：梯度可加，模型状态仍复制

从SGD公式到分工

原页以梯度求和写更新。若目标是全局平均损失 L=(1/B)Σ_i f(x_i;θ)，将B样本均分到d张卡，每卡B/d个，局部平均梯度g_r=(d/B)Σ局部∇f。全局梯度就是(1/d)Σ_rg_r。AllReduce(sum)后除d可保持与单卡平均损失一致。若各卡有效token数不同，应按有效token总数加权；直接平均各卡局部均值通常不正确。

编者例子与规模分析

8个样本分给2卡，各自4个。两卡局部平均梯度为[2,4]、[6,8]，全局平均为[4,6]。从同一θ出发、使用同一优化器状态和步数，各卡更新后仍一致，因此下一批又可独立计算。前反向工作理想上分到1/d，但每步仍需交换长度P的梯度；环发送量约2(d−1)Pq/d。局部批量越小，计算越难覆盖固定梯度通信。

读第16页的复制图

每张GPU上都是完整模型，差别只在数据。参数、梯度和优化器状态都可复制；所以“加卡”不能消除单卡装不下模型的障碍。这里所说的相同更新以相同初值、正确同步、匹配的随机/优化器处理为前提，实际浮点计算不承诺逐位等价。

本节参考（完整书目见讲义末尾）：[slides] · [nccl]

What's wrong with naïve data parallelism? - Memory

Our memory situation is actually *terrible*.

Depending on our precision..

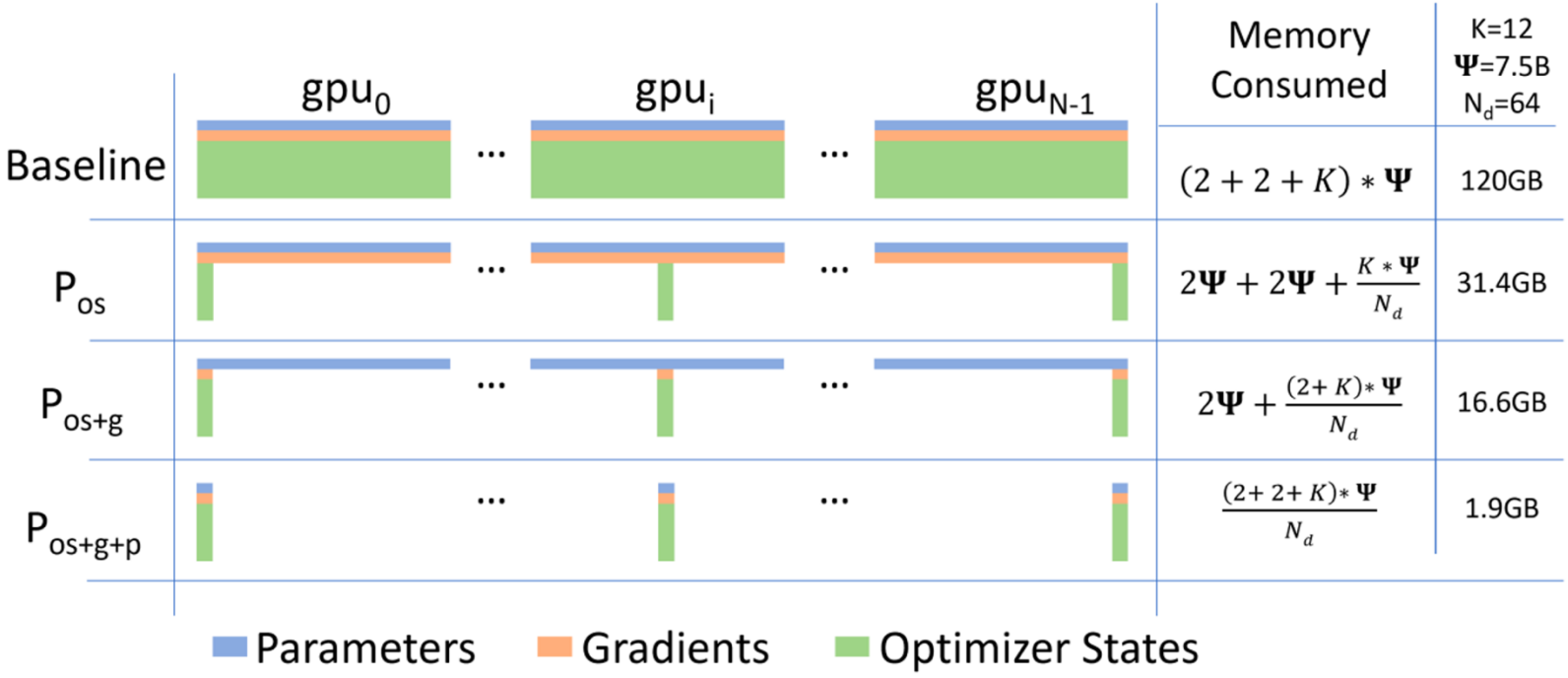
We need 5 copies of weights and 16 bytes per param!

- 2 bytes for FP/BF 16 model parameters
- 2 bytes for FP/BF 16 gradients
- 4 bytes for FP32 master weights (the thing you accumulate into in SGD)
- 4 (or 2) bytes for FP32/BF16 Adam first moment estimates
- 4 (or 2) bytes for FP32/BF16 Adam second moment estimates

"Optimizer state"

ZeRO – solving the memory overhead issue of DP

Core idea: split up the expensive parts (state) and use the reduce-scatter equivalence.



第 8 讲 · 原页 17–18 · 中文讲解

先列状态账本，再理解ZeRO的切分图

16字节从哪里来

P个参数采用本页混合精度Adam约定：模型参数2P字节、梯度2P、FP32主权重4P、第一矩m为4P、第二矩v为4P，总计16P。主权重用于高精度累积小更新，再转换成计算用低精度权重；m追踪梯度平滑趋势，v追踪平方梯度尺度。“5 copies”指五类同形状态，并非五份同精度权重。具体实现可以采用不同梯度/矩精度，16不是自然常数。

统一符号与分片账本

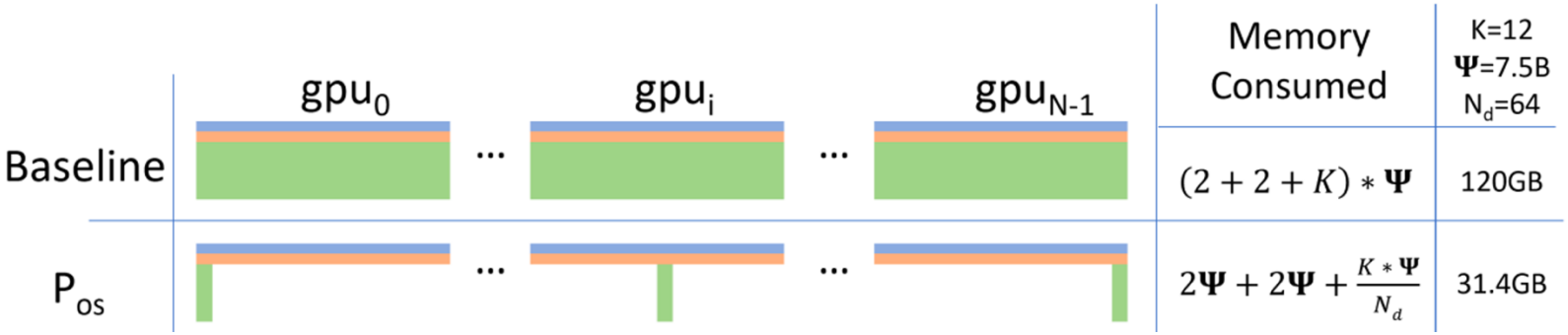
记参数w字节/个、梯度g字节/个、优化器状态K字节/个，K在本节包括主权重和两个矩。组大小d，均匀分片且忽略临时区时：DDP=(w+g+K)P；ZeRO-1=(w+g+K/d)P；ZeRO-2=(w+(g+K)/d)P；ZeRO-3=(w+g+K)P/d。本页w=g=2、K=12。原图蓝/橙/绿分别表示参数/梯度/优化器状态，逐行变短表示去掉复制冗余。

编者复算原图

P=7.5×10^9、d=64时，DDP为120 GB，ZeRO-1为31.40625 GB，ZeRO-2为16.640625 GB，ZeRO-3为1.875 GB，分别对应图中120、31.4、16.6、1.9。最后一个小数字只是常驻状态；完整层参数、激活和通信缓冲依然会临时占显存。

本节参考（完整书目见讲义末尾）：[slides] · [zero]

ZeRO stage 1. optimizer state sharding



High level idea:

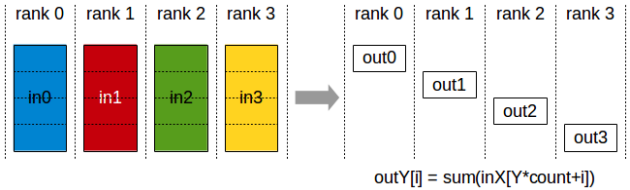
- Split up the optimizer state (first + second moments) across GPUs
- Everyone has the parameters + gradients

Each worker is responsible for updating a subset of params (corresponding to their slice)

ZeRO stage 1. how it works

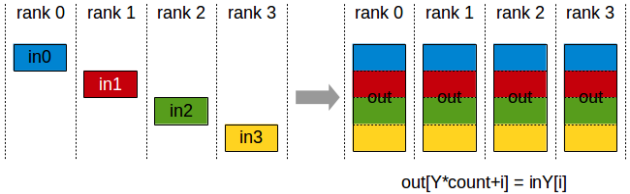
Step 1. Everyone computes a full gradient on their subset of the batch

Step 2. ReduceScatter the gradients – incur #params communication cost



Step 3. Each machine updates their param using their gradient + state.

Step 4. All Gather the parameters – incur #params communication cost



Comparing ZeRO stage 1 and naïve data parallel

	Naïve DDP	ZeRO stage 1
Communication primitive	One all-reduce (gradients)	One reduce scatter (send gradients) + all gather (collect params)
Communication cost	2* # params	2* # params
Memory	(4+K) * #params	(4+K/Ngpu) * #params

ZeRO stage 1 is *free* (in the bandwidth limited regime) memory wins

第 8 讲 · 原页 19–21 · 中文讲解

ZeRO-1：分配更新责任，复用原来的通信预算

一步步跟踪状态

每卡持有完整参数和本地批量产生的完整梯度，但只存1/d的主权重和Adam矩。反向后ReduceScatter把相应参数的全局梯度送到负责卡；每卡用自己的矩和主权重更新该片参数；随后AllGather更新后的低精度参数，让下一次前向开始时每卡又有相同完整模型。注意第二步和最后一步传的是不同对象：前者梯度，后者新参数。

编者两卡例子

θ=[1,2,3,4]，两卡局部梯度分别[1,1,1,1]和[3,3,3,3]，按平均目标归约得到[2,2,2,2]。rank0负责前两个参数，rank1负责后两个。若用η=0.1的SGD简化更新，得到片段[0.8,1.8]与[2.8,3.8]，AllGather后两卡都持有[0.8,1.8,2.8,3.8]。Adam虽还需m、v，却同样可按参数片段局部更新；梯度裁剪等全局统计需另行同步。

为什么说“免费”以及哪里不免费

若梯度和参数通信都用q字节/元素，发送量为(d−1)Pq/d+(d−1)Pq/d，与环AllReduce相同。显存却从(4+K)P降到(4+K/d)P。这里“免费”仅指主导带宽项，不包括分桶、更新调度、延迟、对齐开销；若梯度用FP32而参数用BF16，应分别计算qg和qw，不能机械照搬2P。第19页文字只提矩状态，读第21页K时还要计入主权重。

本节参考（完整书目见讲义末尾）：[slides] · [zero]

ZeRO stage 2. the simple extension to gradient sharding



Emboldened by our success, let's shard even more stuff

High level idea

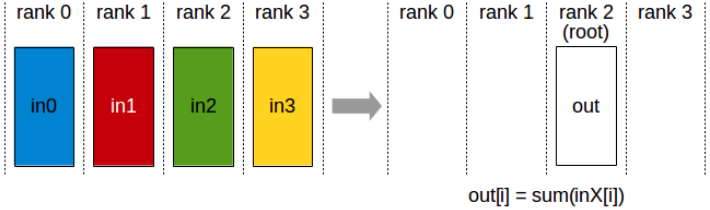
- Also keep the gradients (pink slices) sharded across the machines.
- Use the same (rough) tricks as stage 1.

Complexity – we can never instantiate a full gradient vector, but each worker must compute a full gradient (since we're data parallel)

ZeRO stage 2. how it works

Step 1. Everyone incrementally goes backward on the computation graph

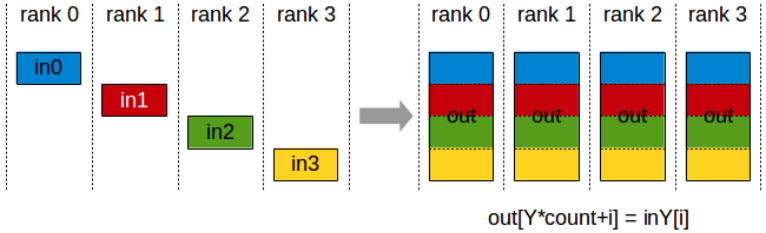
Step 1a. After computing a layer's gradients, immediately reduce to send this to the right worker



Step 1b. Once gradients are not needed in the backward graph, immediately free it.

Step 2. Each machine updates their param using their gradient + state.

Step 3. All Gather the parameters.



第 8 讲 · 原页 22–23 · 中文讲解

ZeRO-2：梯度产生后及时归约并释放

解决看似矛盾的一句话

每卡仍对自己的数据做整个模型的前反向，所以每个参数都可能产生本地梯度；“梯度分片”说的是持久存储，而不是只求1/d的模型梯度。沿反向图走到某层或某个bucket（通信分桶）时，梯度一旦就绪就归约给拥有者，释放不归自己保管的部分。原页“不能实例化完整梯度向量”应理解为避免整模型梯度长期同时驻留，当前桶的完整局部梯度仍可存在。

生命周期与公式

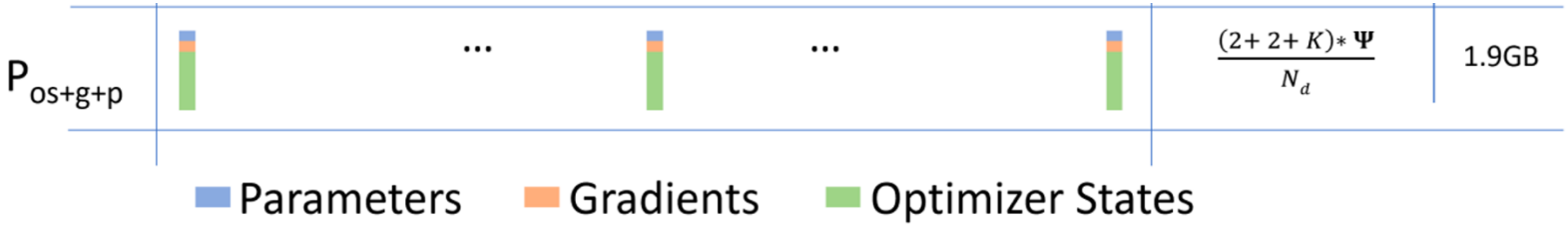
普通ZeRO-1等待所有本地梯度齐全时，完整梯度占gP；ZeRO-2让梯度存储随就绪桶周转，常驻状态变为wP+(g+K)P/d。每个参数的梯度仍归约一次，更新后的参数仍汇集一次，所以单次同步更新的主导通信量与ZeRO-1相同。必须等通信完成且反向不再引用缓冲后才能复用内存。

小桶、大桶与梯度累积

小桶更早通信、释放更及时，但集合操作次数增多；大桶降低启动次数却抬高峰值。m个微批累积时，若每个微批都ReduceScatter，流量可能随m增长；若延迟同步并存全梯度，又会牺牲分片带来的显存节省。后文所有“每步2P”都对应指定同步策略，不能不看实现就用于梯度累积。

本节参考（完整书目见讲义末尾）：[slides] · [zero]

ZeRO stage 3 (aka FSDP) shard *everything*



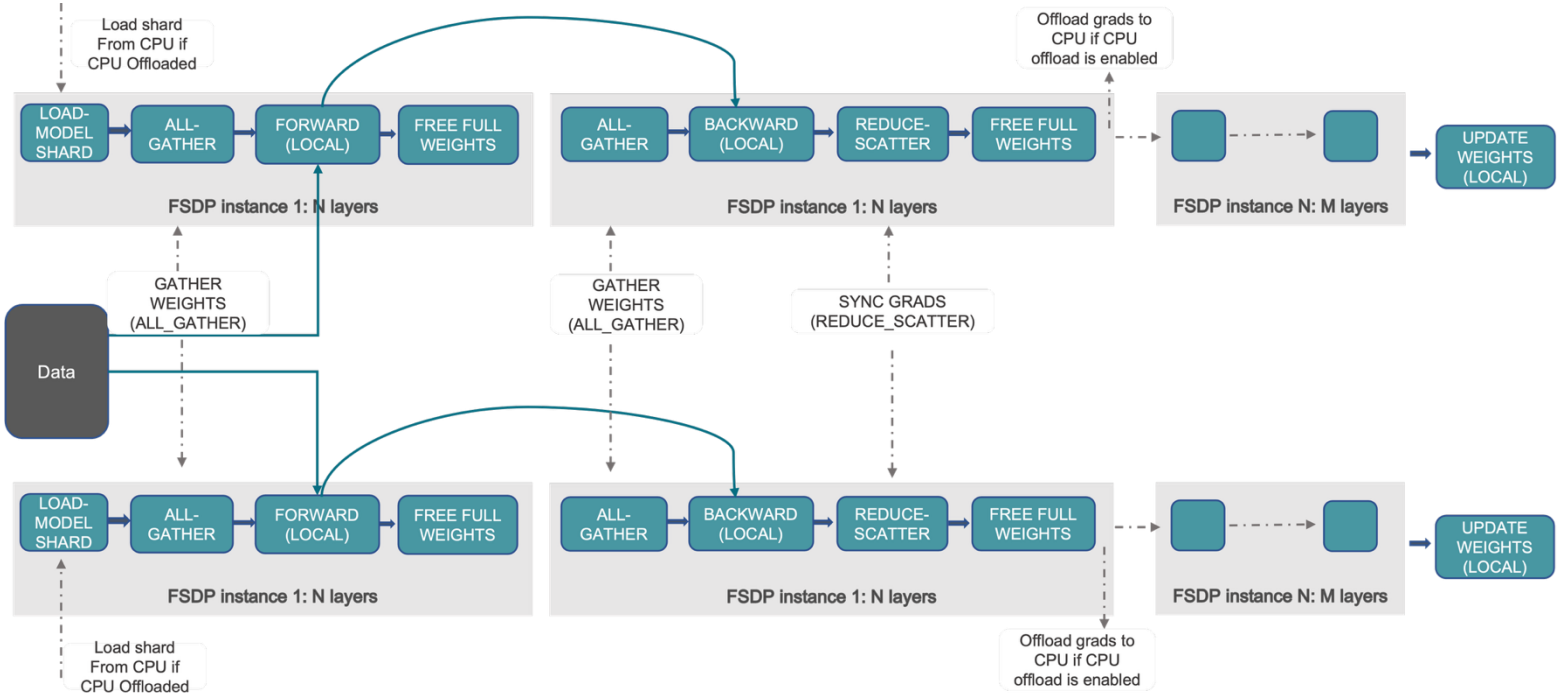
We've gotten almost everything for free so far.. lets try to solve *all* our memory issues

High level idea

- Shard everything – incl parameters!
- Use the same 'incremental communication / computation' ideas
- Send and request parameters on demand while stepping through the compute graph.

Is it possible to do this with low overhead?

ZeRO stage 3 (aka FSDP) how it works (baby version)



Communication cost – 2 all gather (#param), 1 reduce-scatter (#param).

https://pytorch.org/tutorials/intermediate/FSDP_tutorial.html

Actual picture of how FDSP / ZeRO stage 3 works

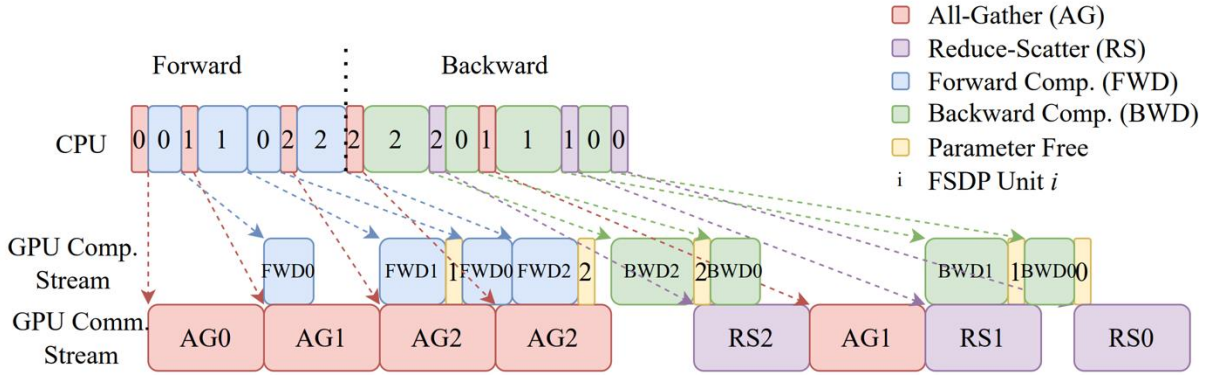
Let's walk through a FSDP example to see some important ideas

Incremental computation / communication

- Parameters / gradients are requested / sent and then immediately freed

Overlapping communication and computation $(W_1W_0 + W_2W_0)x = y$

- The all-gathers happen all at once while forward happens, masking the comm cost.



https://arxiv.org/pdf/2304.11277.pdf

第 8 讲 · 原页 24–26 · 中文讲解

ZeRO-3与FSDP：完整权重只在需要时出现

顺着第25页箭头走

常驻只留参数、梯度和优化器状态的各自片段。处理一个FSDP单元时，先AllGather其参数，完成本卡数据的前向；若前向后reshard，就释放完整参数。反向来到该单元前再AllGather一次；求出梯度后ReduceScatter，保留自己负责的梯度片段并释放完整参数。最后每卡更新自己的状态片段。图里的CPU offload是可选机制，不是FSDP定义本身。

编者推导：3P和峰值显存

在每个参数前向/反向各汇集一次、梯度归约一次、参数与梯度通信精度相同的前提下，每卡发送约3(d−1)Pq/d，比DDP的2(d−1)Pq/d多50%。更一般是(d−1)P(2qw+qg)/d。若前向后保留完整权重，反向无需再汇集，流量可降但显存会上升；重计算、重复调用模块和分桶也会改变次数。因此“FSDP=3P”是此调度的记账式。

如何读第26页时间线

横轴是时间，计算stream与通信stream分行：当前单元在计算时，可预取下一个单元的参数；反向同时发送已完成的梯度。不是整模型所有AllGather瞬间同时完成，也不是任何通信都能被遮住。预取增加在途完整参数，峰值更接近“常驻分片+当前完整单元+预取单元+激活+工作区”，不能仅看16P/d。单元太大放不下，太小则延迟显著。

术语边界

ZeRO-3是三类状态都分片的思想，FSDP是实现这一类训练的框架机制，不同版本和配置不等同于同一个通信轨迹。第26页FDSP是原页拼写问题，本文统一写FSDP。分片优化解决模型状态冗余，但每卡仍处理完整网络的本地数据，激活没有因此自动按d缩小。

课堂补充 · 00:26:14–00:27:49

教师在答疑中区分FSDP与按层放GPU：FSDP里每卡仍从头到尾计算自己的样本，只是参数按需汇集。另一问是为何通信不再乘层数：各层参数量相加已是全模型P，次数随层数增加而单次消息变小；延迟次数可能增加，但不能把总字节3P再乘层数。此处为字幕核对后的转述。

本节参考（完整书目见讲义末尾）：[slides] · [zero] · [fsdp] · [fsdp-doc] · [captions]

What's the point?

Distributed data parallel costs 2*# param communication

What about ZeRO?

- **Zero stage 1** is 2*# param – it's free! – you might as well always do it
- **Zero stage 2** is 2*# param – it's (almost) free (ignoring overhead)
- **Zero stage 3** is 3*# param – 1.5x comm cost, but that's not bad! (ignoring latency..)

This is also conceptually very simple – write a FSDP block wrapper.

ZeRO in practice: will it fit?

Pure BF16 training (with Kahan summation), is viable and optimizer states are less beefy. Let's say BF16 for everything but the master weights – 12 bytes per param

On a 8X A100 80G..

	Max size (params)	Formula for B/param
Baseline	6.66.. B	12
Zero stage 1	16 B	5
Zero stage 2	24.62 B	2 (param) + (10 (grad+state))/8)
Zero stage 3	53.33 B	12/8

第 8 讲 · 原页 27–28 · 中文讲解

通信与显存汇总：复算8张80 GB卡的容量表

先换精度，再代公式

第17页K=12是FP32主权重与两个FP32矩；第28页另设“除主权重外均BF16”：w=2、g=2、主权重4、两个矩各2，所以K=8，总计12字节/参数。d=8时，DDP=12；ZeRO-1=4+8/8=5；ZeRO-2=2+(2+8)/8=3.25；ZeRO-3=12/8=1.5，单位都是每卡每参数字节。这正好复现原表，不能把上一页的K=12带入后再认定表错了。

编者容量手算

仅把每卡80×10^9字节全部用于这些状态，上界分别80/12≈6.67B、80/5=16B、80/3.25≈24.62B、80/1.5≈53.33B参数。8×80/12=53.33B解释了完全分片理想上界。真实模型上限会更低，尤其ZeRO-3还有完整计算单元与激活。80GB与80GiB也不相同，必须统一单位。

数值与性能边界

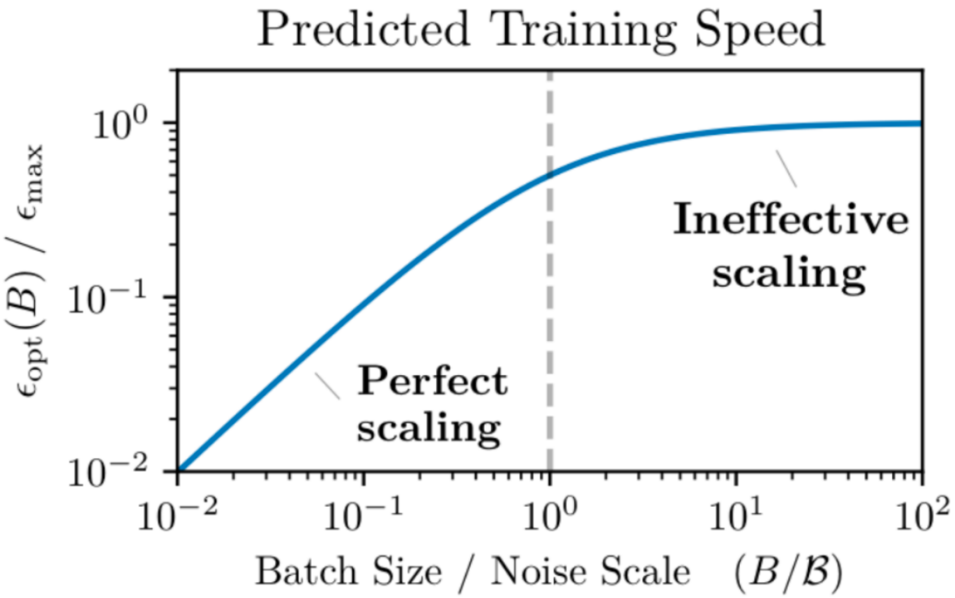
原页提Kahan补偿求和，动机是避免低精度累积丢失很小的更新；它不是“BF16训练必然稳定”的证明。这里的12字节只是给定状态清单的预算，不额外假定一份补偿缓冲免费。原页27的2P、2P、3P总结依赖大消息、同通信精度与既定调度；具体实现选择应同时量峰值和关键路径，而不是仅凭“free”一词。

本节参考（完整书目见讲义末尾）：[slides] · [zero] · [fsdp]

Issues remain with data parallel – compute scaling

With data parallel**, #machines < batch size** (and near this, comm overhead is high)

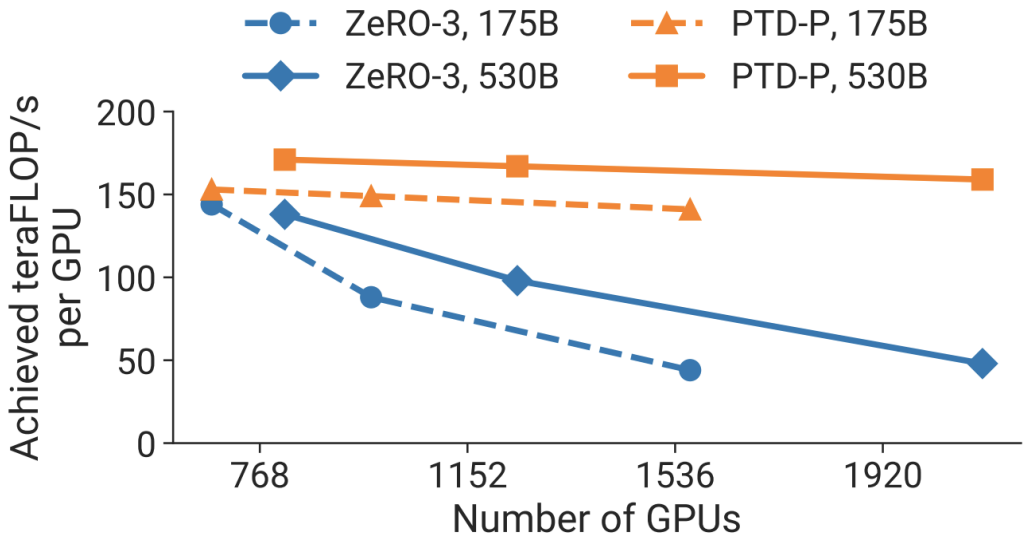
And there's diminishing returns to batch sizes



Issues remain with data parallel – models don't fit

Zero stages 1 and 2 don't let you scale memory

Zero stage 3 is nice in principle, but *does not reduce activation memory*



Better ways to split up the model is needed…

Beyond data parallel – model parallelism

Scaling up in memory (without changing batch size) with model parallelism

What model parallelism is..

- It splits up the parameters across GPUs (like zero3)..
- But communicate activations (while zero3 sends params).

We cover three different types of parallelism

1. Pipeline parallel
2. Tensor parallel (+ Sequence parallel)
3. Expert parallel

These correspond to three different ways of cutting up the model.

第 8 讲 · 原页 29–31 · 中文讲解

为什么还需要模型并行：批量与激活的限制

固定批量下的强扩展

B固定时，每个DP副本拿到B/d个样本。d增加使计算减少，但每次更新的梯度通信规模没有按1/d消失，所以收益递减。把B随d一起增加虽然可维持本地工作量，却会改变优化过程；第29页曲线把batch相对于梯度噪声尺度画在横轴，表示增大batch的训练步数收益会逐渐饱和，不是所有模型共享一个固定临界batch。

容量瓶颈还剩什么

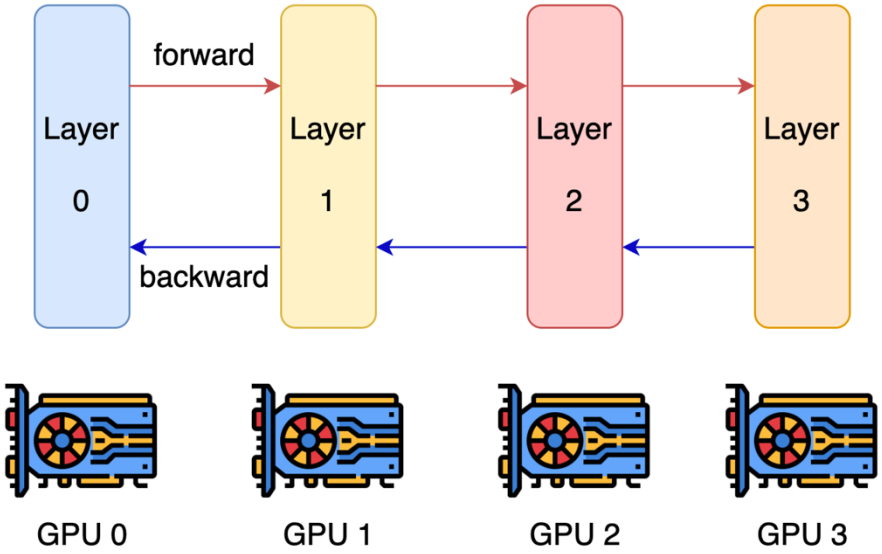
ZeRO-1/2确实节省显存，但仍分别有(w+g)P与wP的复制下界，不能随d无限线性扩展。ZeRO-3去除了常驻状态冗余，仍需当前完整单元与本地数据激活。固定本地batch时，这部分激活不会自动除d；固定全局batch时，局部batch减少当然可能让激活减少。原页“does not reduce activation memory”说的是分片机制本身。

读第30页对比图与第31页分类

图中ZeRO-3随卡数增加每卡吞吐下降，而组合模型并行曲线更平：在该实验的固定全局batch和硬件上，通信主导更严重。它不是FSDP在所有场景都差的证明。模型并行把同一样本的计算分给多卡：PP切深度，TP切矩阵，EP切专家；主要交换中间激活，与ZeRO反复汇集权重形成不同的代价结构。

本节参考（完整书目见讲义末尾）：[slides] · [megatron]

Layer-wise parallel



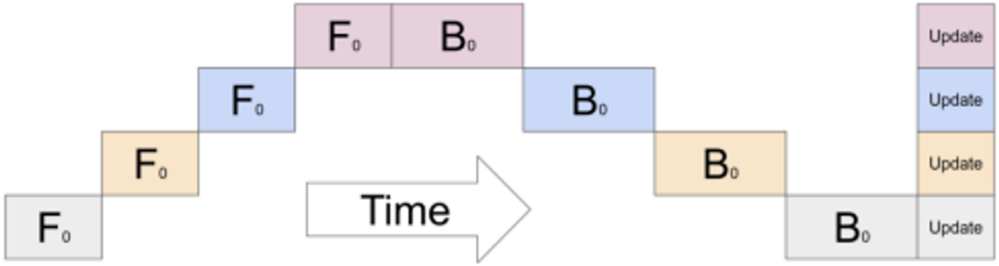
Layer-wise parallel cuts up layers, assigns some subset to GPUS.

Activations and partial gradients are passed back and forth

What's wrong with layer-wise parallel

Utilization of layer-wise parallelism is *terrible*..

With n gpus, each gpu is active $\frac{1}{n}$ of the time.



Each GPU is idling most of the time, waiting for the backward pass to propagate back

第 8 讲 · 原页 32–33 · 中文讲解

按层放卡：先看懂朴素串行时间线

前反向必须等待哪些信息

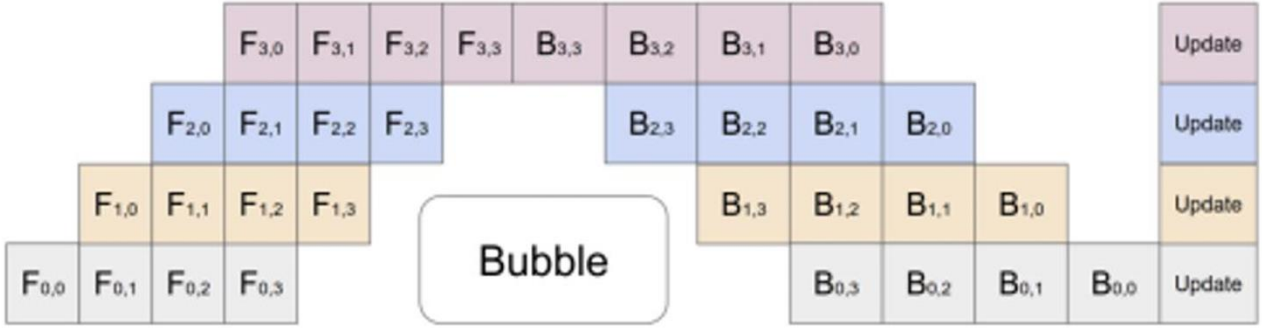
设4层分别放GPU0至GPU3。前向把x逐层变成激活，反向从最后一层损失出发，把对上一层输出的梯度逐段传回。每卡保留自己的权重和该层需要的激活，参数容量可以按深度切开；但只有一个微批时，后面的卡必须等前面的激活，前面的卡又要等后面的反向梯度。第33页横向时间线中的空白就是空闲。

编者推导**1/p**的前提

设p个阶段负载相同，每阶段前向f时间、反向r时间，忽略通信与更新。单微批总时长p(f+r)，每卡只忙f+r，所以平均利用率1/p。p=4则仅25%。这不是GPU算子内部利用率，而是整步时间内设备的调度占用率；阶段不均衡、通信与额外等待通常让情况更差。解决方向是在某卡等待这一微批时，让它处理别的微批。

本节参考（完整书目见讲义末尾）：[slides]

A solution: pipeline parallel



Solution: Pipeline-parallel.

Process ‘micro-batches’ (in this case, 4).

Send off the first microbatch and start computing the second.

The ratio of bubble time to useful compute is .. $\frac{n_{stages}-1}{n_{micro}}$ so we need a big batch size!

第 8 讲 · 原页 34–34 · 中文讲解

流水线微批：气泡比、空闲比例与利用率

读图并区分批量

图中每种颜色代表一个阶段，F与B分别是前向和反向，下标区分阶段/微批。将一个副本的一批样本拆为m个微批，每微批b个样本；前一个微批离开阶段0后，阶段0可继续处理下一个。整个更新仍只在全部微批的梯度累积完成后执行。若有d个DP副本，全局批量B=dmb，微批不是额外的优化器更新次数。

编者推导：填充与排空

在图示的先全前向后全反向（GPipe式）调度中，假设各阶段前向耗时f、反向耗时r且阶段均衡，忽略通信/更新。m个微批完成前向需(m+p−1)f，反向需(m+p−1)r，总T=(m+p−1)(f+r)。每卡有用工作是m(f+r)，空闲等效时间为(p−1)(f+r)。所以原稿气泡/有用计算=(p−1)/m；气泡/总时长=(p−1)/(m+p−1)；利用率U=m/(m+p−1)。三者不能混用。

编者手算与限制

p=4、m=4时，气泡/计算为3/4=75%，但空闲占总时长3/7≈42.9%，利用率4/7≈57.1%。m增到16，利用率16/19≈84.2%。要达至少90%，需m≥9(p−1)，p=4时至少27个微批。该公式针对理想均衡调度；最慢阶段、通信不重叠、优化器时间和其他调度会改变实测值。

本节参考（完整书目见讲义末尾）：[slides] · [megatron]

Why pipeline parallel?

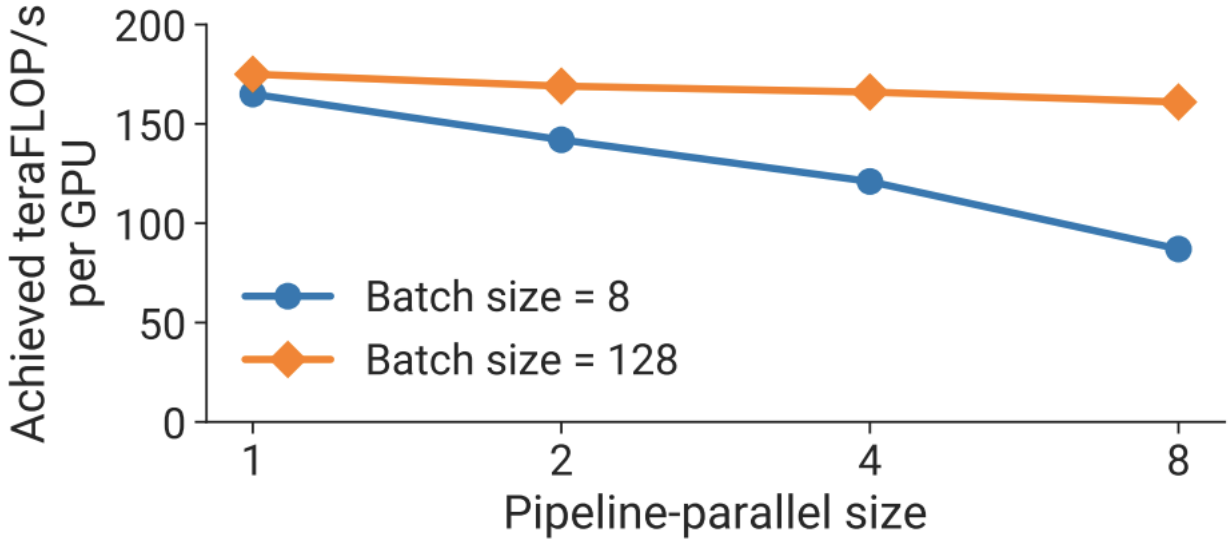
Pipelines seem terrible. Why do we do it?

1. Pipelines save memory (compared to DDP)

2. Pipelines can have good communication properties (compared to FDSP) – it depends only on activations ($b \times s \times h$) and is *point to point*

Generally, we will use pipelines on slower network links (i.e. inter-node) as a way to get better memory-wise scaling.

Pipeline performance is highly dependent on batch size



Batch sizes are key to hiding the bubble – otherwise pipeline rapidly degrades perf

第 8 讲 · 原页 35–36 · 中文讲解

为什么PP仍有用：少量边界通信换容量

边界消息与逐层集合操作的区别

一个阶段只把末端激活传给下一阶段。假设边界张量形状为[b,s,h]且未做额外分片，前向单方向一次是bsh个元素，即qbsh字节；反向沿同一边界传同形输入梯度，再传一次。故单边界一微批前反向为2qbsh字节，若统计某个内部阶段的两个相邻边界，还需分别计入。PP无须在每一层都把全组卡唤起做集合通信。

编者例子与图意

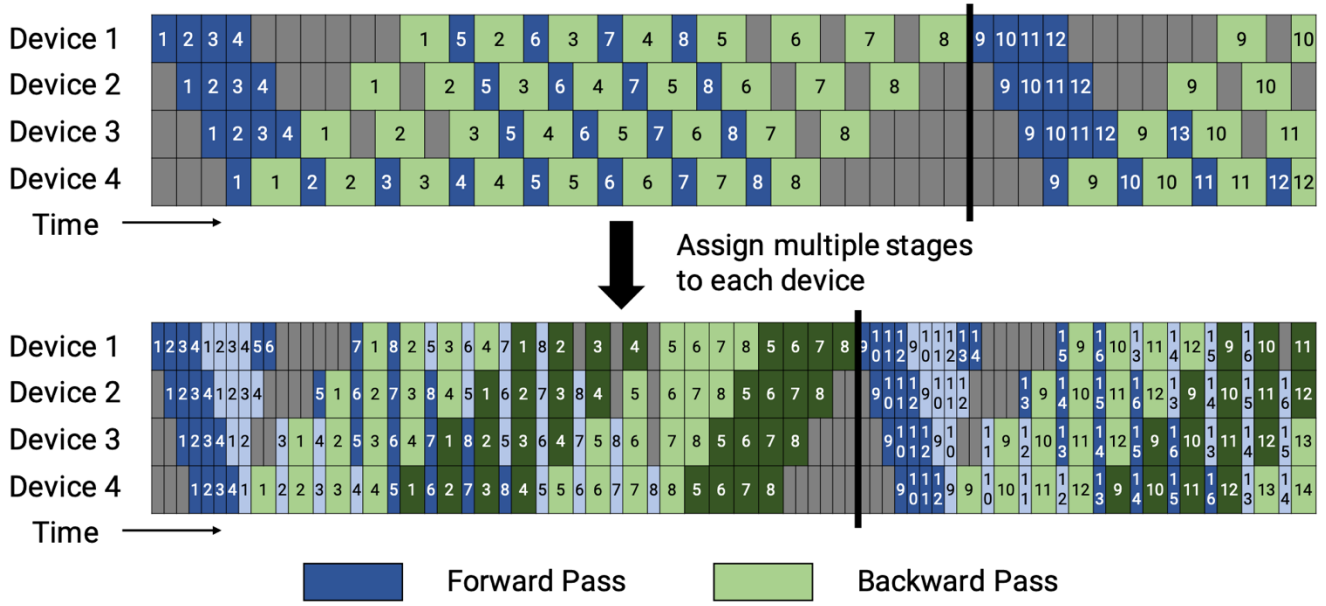
b=1、s=2048、h=4096、q=2时，单方向边界是16 MiB，一次前反向跨该边界共32 MiB。层数增多但分界不变时，边界消息尺寸不按层数增加；不过计算、参数和激活当然增加。第36页横轴为PP阶段数，较小batch曲线下降更快，正对应m相对p过小时的气泡。

不是无代价地增大**m**

在B固定且d固定时，增加m意味着缩小b。过小b可能让矩阵乘法变窄、内核效率下降，且消息更多更碎。先全前向再反向需保留较多微批激活；1F1B调度在填充后交替做前向和反向，能较早释放激活，通常改善缓冲需求，但仍有填充/排空气泡。PP是容量、网络与调度的折中，常适合跨节点连接。

本节参考（完整书目见讲义末尾）：[slides] · [megatron]

Trading communication bandwidth for utilization



Some more crazy pipeline patterns can improve utilization, but at the cost of bandwidth

‘Zero bubble’ pipelining

Split up backwards into two parts

1. Backpropagating activations (z,x)
2. Computing weight gradients (W)

The second part can be done whenever

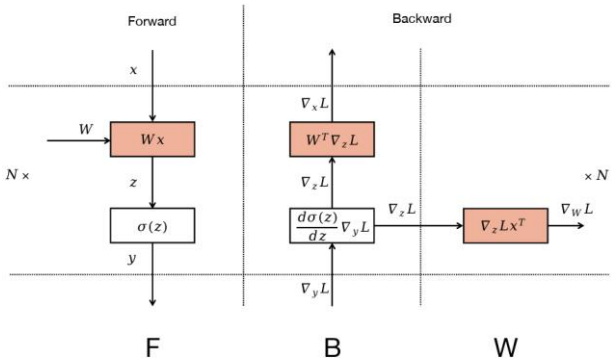


Figure 1: Computation Graph for MLP.

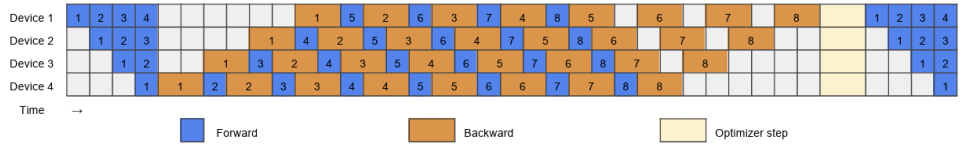


Figure 2: 1F1B pipeline schedule.

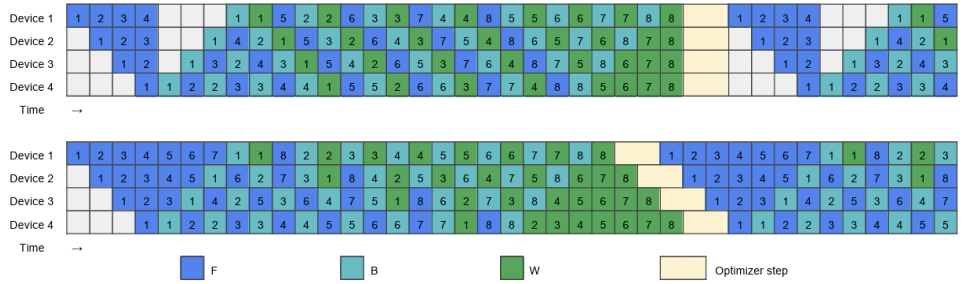


Figure 3: Handcrafted pipeline schedules, top: ZB-H1; bottom: ZB-H2

第 8 讲 · 原页 37–38 · 中文讲解

更细的流水线调度：交错与近零气泡

第37页：一张卡不必只负责一段

图下半把同一物理设备的工作分成多个较短虚拟阶段，并交错安排微批。短阶段可以缩短填充/排空的关键路径；但同一微批会跨设备更多次，增加通信次数、激活周转与调度复杂度。理想分析中每卡v个均衡虚拟阶段可把传统气泡比例约缩小v倍，前提是通信与阶段切换的新增代价没有抵消收益，并非无条件加速。

第38页：反向可分成两个任务

编者推导：线性层Z=XW，设G=∂L/∂Z，则输入梯度dX=GW^T，权重梯度dW=X^TG。前一个阶段等待的是dX，不必等待本层dW完成；因此可以优先计算并发送dX，把dW安排到原本空闲的时隙。图中不同反向颜色表达的正是两类计算的拆分。

“whenever”的真实约束

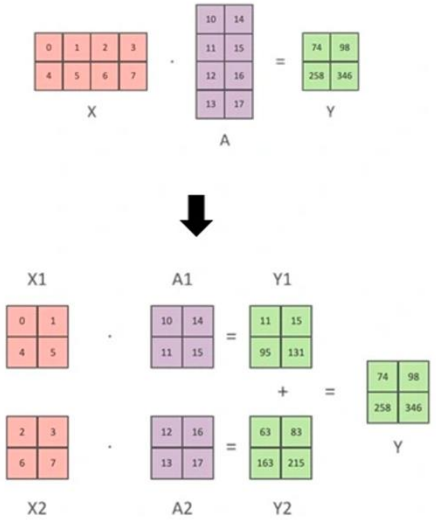
dW必须在X和G均可用之后、该次优化器更新之前完成；推迟它要保存输入和输出梯度，会增加内存。近零气泡还依赖阶段负载、内存预算、通信和更新的同步条件。不同层的前向、dX、dW耗时也不一定相等。名字里的zero是调度目标，不意味着网络等待、数据读取或故障恢复都为零。

本节参考（完整书目见讲义末尾）：[slides] · [megatron] · [zero-bubble]

Model parallel along the width axes

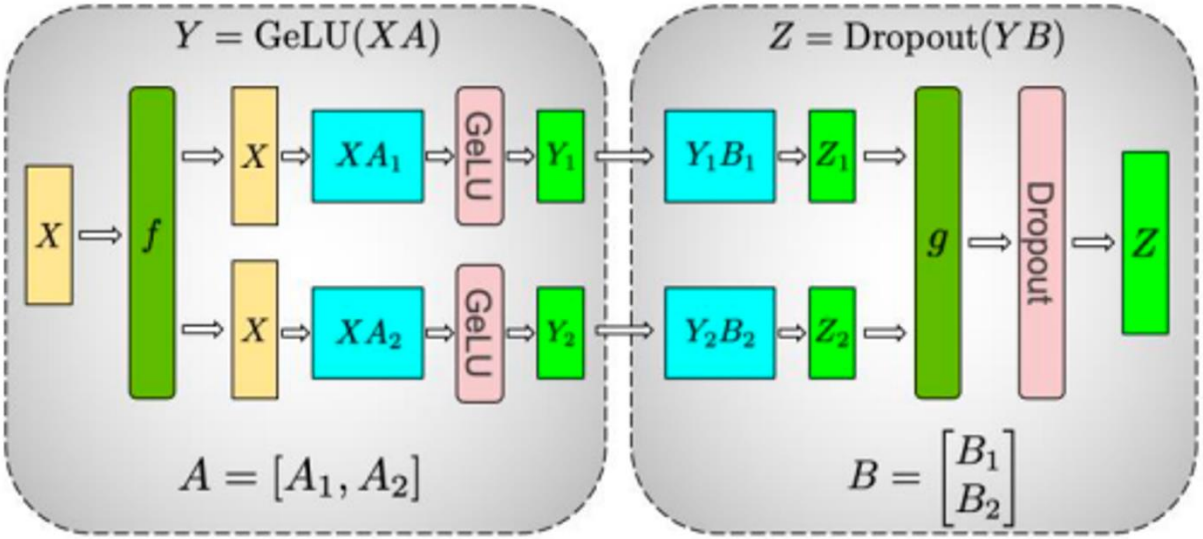
Are there model parallel schemes with better utilization?

We can think of pipeline parallel as cutting up along depth. What about width?



Simple matrix multiplication observation: decompose into submatrices, add partial sums

Tensor parallel – GPUs have submatrices



Assign columns (A1, A2) and rows (B1, B2) to separate GPUs.

- In the forward pass, f is the identity, and g is an all-reduce.
- In the backward pass, f is an all-reduce, g is the identity.

第 8 讲 · 原页 39–40 · 中文讲解

TP的代数核心：列切分后接行切分

编者从形状推导MLP

把[b,s,h]展平为X，形状[bs,h]。以4h中间维的MLP为例，A形状[h,4h]，B形状[4h,h]，Y=GeLU(XA)，Z=YB。两卡将A按输出列切成A_1,A_2，各[h,2h]；卡i计算Y_i=GeLU(XA_i)，形状[bs,2h]。GeLU逐元素，所以对列块分别作用等价于完整计算后切分。B沿输入行切成B_1,B_2，各[2h,h]，卡i算Z_i=Y_iB_i，最后Z=Z_1+Z_2。中间Y不必AllGather。

编者小矩阵：先学会加局部积

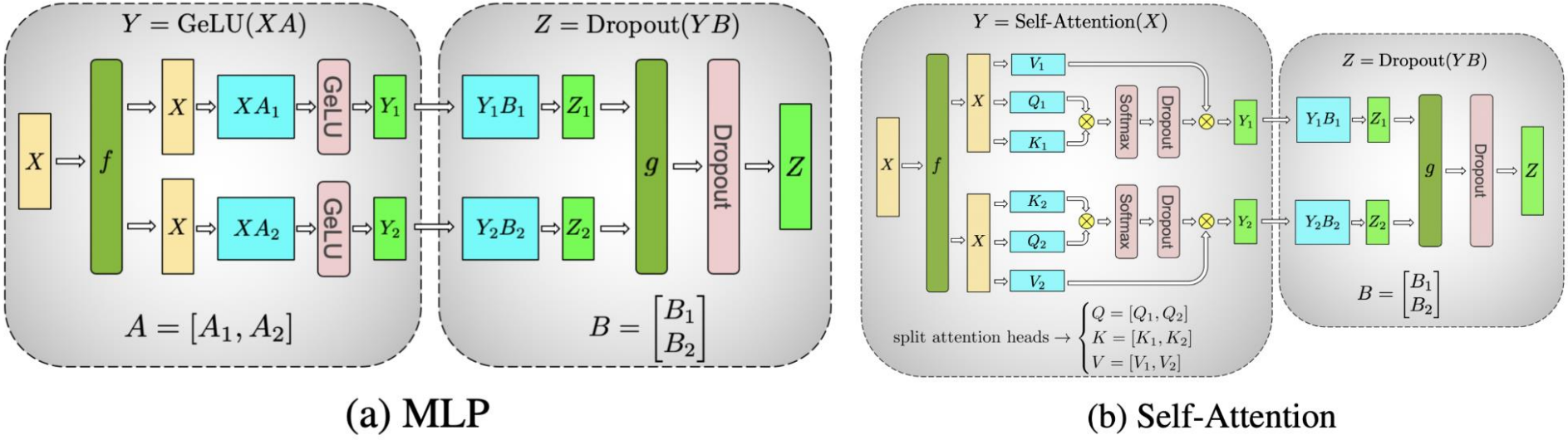
取X=[1,2]，A为2×2单位阵，B=[[1,2],[3,4]]，暂把GeLU换成恒等以便手算。卡0得到Y_0=[1]、Z_0=[1,2]；卡1得到Y_1=[2]、Z_1=[6,8]；AllReduce求和后Z=[7,10]，与XAB一致。真实GeLU保留逐元素性质；反过来先把矩阵积按求和项切开，再在各部分上做GeLU通常错误，因为GeLU(u+v)不等于GeLU(u)+GeLU(v)。

f与g的反向含义

图中f在前向把同一X供给两条分支，前向无需归约；反向不同权重分片各给出一份dX贡献，必须求和。g在前向把Z_i相加，让下游取得完整Z；在Megatron的复制/分片张量语义下，下游同一dZ可直接送给各分支，因此g反向是恒等，不再重复求和。这里f/g是一对带布局约定的算子，不能把它理解成所有普通分布式AllReduce的自动求导都固定如此。

本节参考（完整书目见讲义末尾）：[slides] · [megatron]

Row vs Column tensor parallel



(a) MLP

(b) Self-Attention

How do we split up in tensor parallel across a transformer block?

- Columnwise – QKV, up-projection
- Rowwise – Attention output, down-projection
- Replicated – norms, routers, etc

第 8 讲 · 原页 41–41 · 中文讲解

把TP放进Transformer：注意力头也能分工

顺着左右两张图走

左图是上一节的MLP列并行up-projection与行并行down-projection。右图先把Q/K/V投影按输出维切开，让每张卡拥有一组注意力头，独立在完整序列上算这些头的注意力，再把输出投影按输入行切开，求和恢复隐藏维输出。注意：经典多头注意力的头间计算独立，但每个头内部仍需访问全部相应K/V token；TP并没有把注意力的序列依赖删掉。

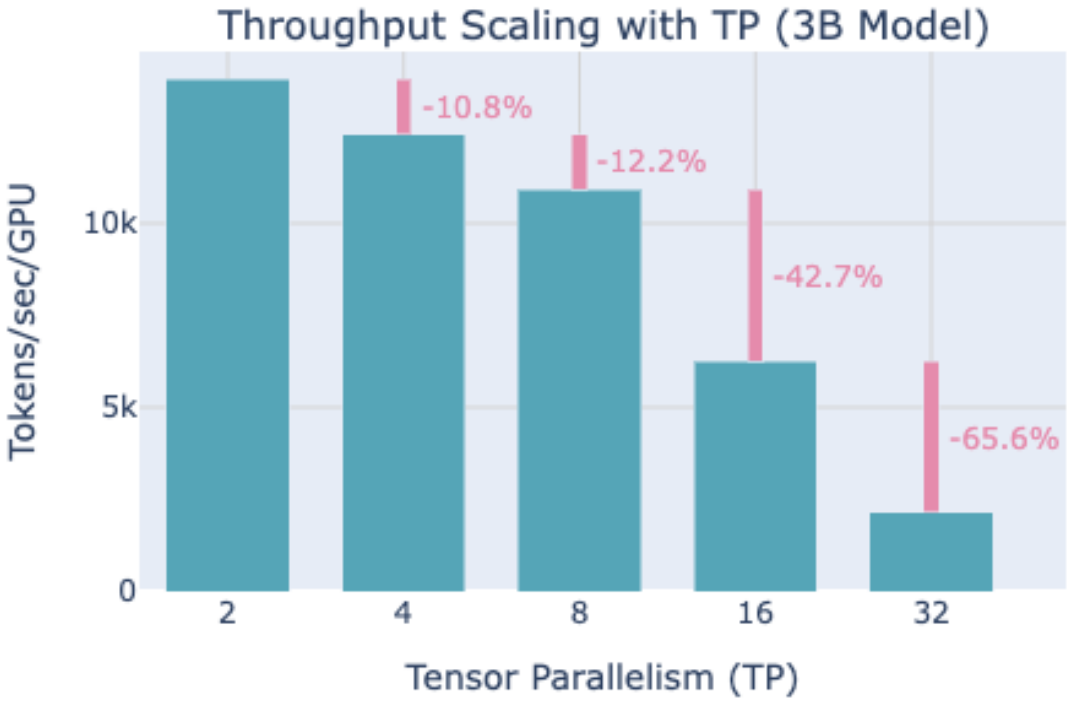
分片的条件与残差

简单分析假设头数和矩阵维度可被t整除，头的分配均衡。GQA/MQA的KV头数较少，可能出现复制KV或限制可用TP度，不能照抄每卡a/t个KV头。LayerNorm/RMSNorm沿每个token的完整隐藏维归一化，常规TP路径中这部分及残差输入常被复制；router也常用复制或专门布局。哪些状态复制需按具体实现查明，这正是后文SP解决的剩余激活。

本节参考（完整书目见讲义末尾）：[slides] · [megatron]

When do we tensor parallel?

On GPUs, tensor parallel within a node (up to 8 GPUs) due to high speed interconnects.



Tensor parallel – pros and cons vs pipeline parallel

How do things compare to pipeline parallel?

Pros – no bubble. If your network is fast enough, there's no waiting for others.
– low complexity – simple to 'wrap' models without major infra changes
– doesn't need large batch sizes to work well

Cons – *much* larger communication than pipeline parallel.

- Pipeline: bsh point-to-point communication per microbatch
- Tensor: $8bsh\left(\frac{n_{devices}-1}{n_{devices}}\right)$ *per layer* and *all-reduce* communication.

Use tensor parallel whenever we have low-latency, high-bandwidth interconnects

第 8 讲 · 原页 42–43 · 中文讲解

TP通信为何重：把8bsh的常数拆开

编者推导：每层四次AllReduce

按本讲经典Transformer块的布局，attention和MLP在前向各有一次输出归约，反向各有一次输入梯度归约，共4次，张量都约bsh元素。环AllReduce每rank发送2(t−1)bsh/t元素，因此前反向总发送量为8bsh(t−1)/t元素；换成字节还需乘通信精度q。这个8来自4次×环的2，不能再把它当8字节/元素。SP、更复杂的GQA布局或融合可能改变操作种类。

编者比较：必须统一范围

b=1、s=2048、h=4096、t=8、q=2时，每层每微批每卡发送112 MiB。相比之下PP的单边界单方向为16 MiB；若比较完整前反向，该边界是32 MiB。TP的112 MiB还会随该卡参与的层数累加，而PP主要发生在阶段边界。不要把TP全层全步流量与PP单条消息直接当作公平加速比。

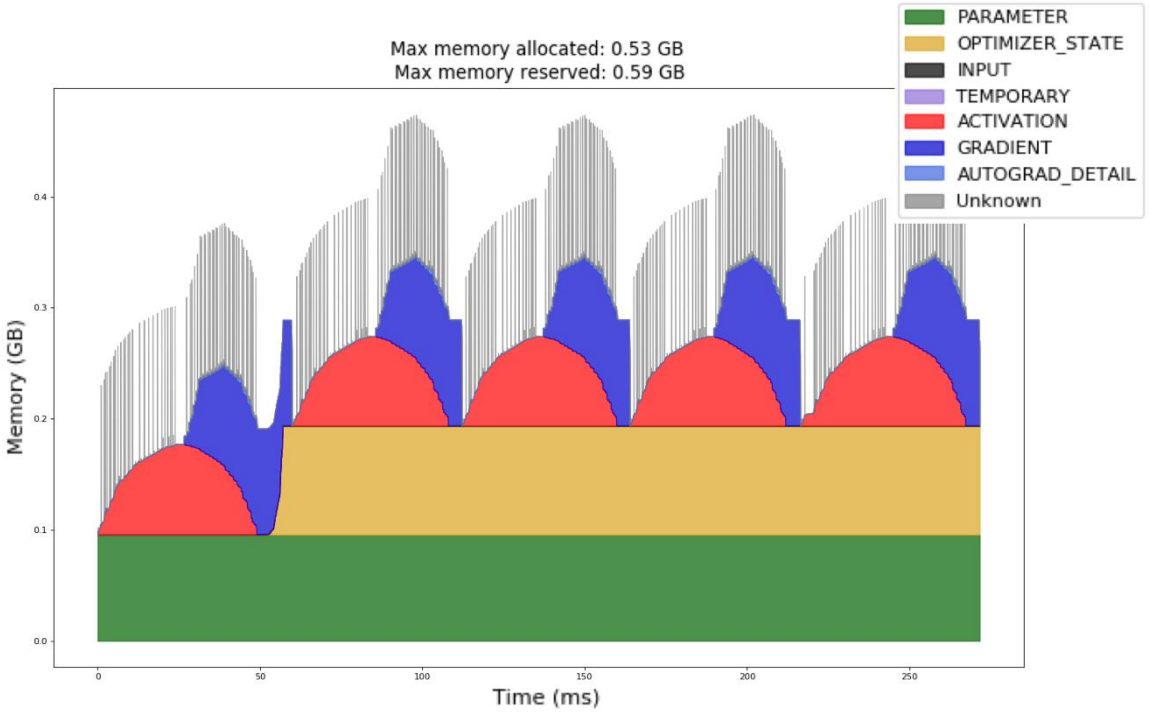
读吞吐图与“无气泡”

第42页纵轴是tokens/sec/GPU，并非全群总吞吐；TP增加时每卡效率下降，跨越高速域后降幅更明显。TP没有PP式填充排空，但仍有同步等待、网络延迟和更小矩阵造成的效率损失。“8卡内”是原系统经验：应按实际高速互联域选择，而不是认为第9张卡在算法上不能TP。

本节参考（完整书目见讲义末尾）：[slides] · [megatron] · [nccl]

A final complexity – memory is dynamic!

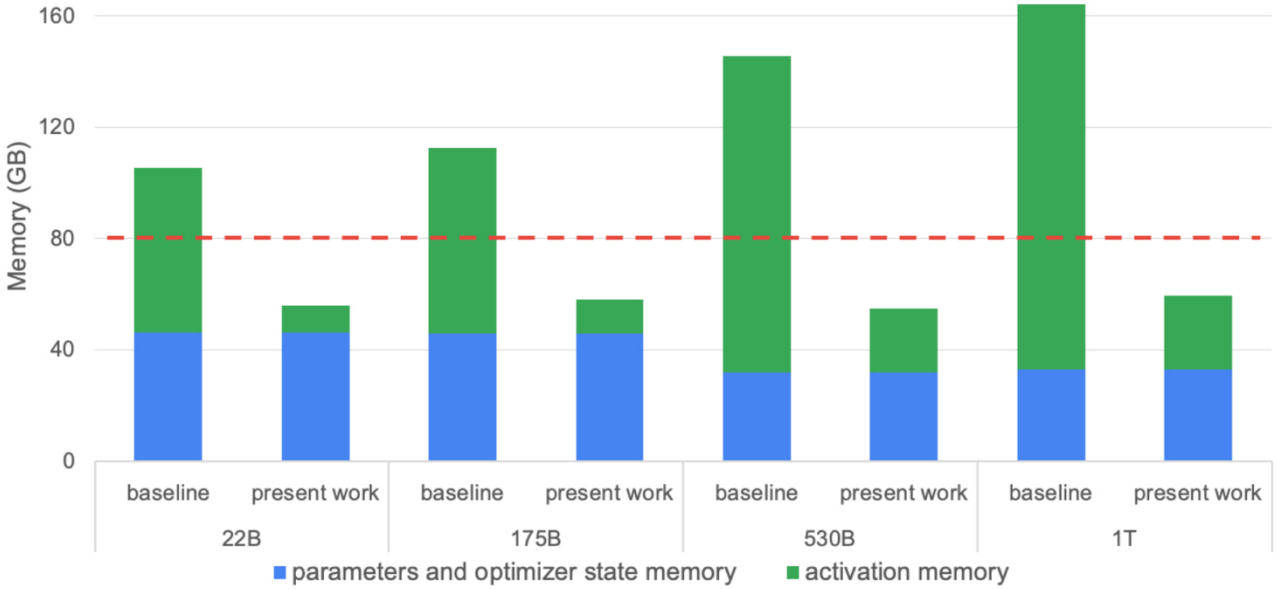
Memory isn't just the static bits, but also activations! This can be big



A final complexity – activation memory

Thus far, we have only really discussed parameter memory.

Tensor and pipeline parallel can linearly reduce those..**but what about activations?**



[Korthikanti et al 2022]

第 8 讲 · 原页 44–45 · 中文讲解

显存随时间变化：峰值由同时存活的对象决定

读显存时间图

第44页底部常驻色带对应参数/优化器等，随前后向起伏的色带含激活、梯度、临时对象和autograd数据。前向不断保存供反向使用的中间值；反向逐层消费并释放，但又产生梯度和临时矩阵。真正的OOM由某一时刻总和的最大值决定，不是各项分别取最小值后相加。allocated与reserved也不同，后者包含分配器缓存空间。

第**45**页柱状图的教训

图中同一模型的baseline与新策略主要差在绿色激活段；红色80 GB线说明参数分片后也可能因激活超容量。训练激活不是推理时的持久KV cache：训练反向需要更多中间数据，且通常以微批为生命周期；推理自回归缓存服务于后续token。表中并列写activation/KV时，要分清两种工作负载。

编者测量思路

估算应把当前层工作区、所有未完成微批的保存激活、参数汇集/梯度通信在途缓冲都纳入时间线。FSDP预取和PP推迟反向都会延长某些对象寿命，尽管单独的状态公式更小。小规模分析时看峰值位置与对象类别，比只记最终稳定显存更有用；本稿不运行多GPU测量。

本节参考（完整书目见讲义末尾）：[slides] · [activation]

What's the activation memory per layer?

Starting point: activation memory needed if storing everything

$$\text{Activations memory per layer} = sbh\left(34 + 5\frac{as}{h}\right).$$

- The $5\frac{as}{h}$ terms come from the quadratic attention terms incl dropout
- As with flash attention, we can drop this term via recomputation

a	number of attention heads	p	pipeline parallel size
b	microbatch size	s	sequence length
h	hidden dimension size	t	tensor parallel size
L	number of transformer layers	v	vocabulary size

Activation under tensor parallel

$$\text{Activations memory per layer} = sbh\left(10 + \frac{24}{t} + 5\frac{as}{ht}\right)$$

Tensor parallel splits out the matrix multiplies in attention + MLP

The remaining **10** term is for the LayerNorm (4sbh), Dropout (2sbh), and inputs to the attention and MLP (4sbh). These terms alone will continue to grow with size

a	number of attention heads	p	pipeline parallel size
b	microbatch size	s	sequence length
h	hidden dimension size	t	tensor parallel size
L	number of transformer layers	v	vocabulary size

第 8 讲 · 原页 46–47 · 中文讲解

激活显存公式：34、10与24究竟是什么

原稿公式与前提

原式每层A=sbh(34+5as/h)=34sbh+5abs^2，单位是字节。这是Korthikanti等人分析的特定GPT式Transformer：标准多头注意力、4h中间维GeLU MLP、常规16bit保存激活与1byte dropout mask，并采用论文的保存策略；不是所有架构的普适常数。a为头数，s为序列长，b为微批大小，h为隐藏维。FlashAttention、GQA、SwiGLU、不同dropout/精度都可能改常数或项。

编者按张量形状拆解

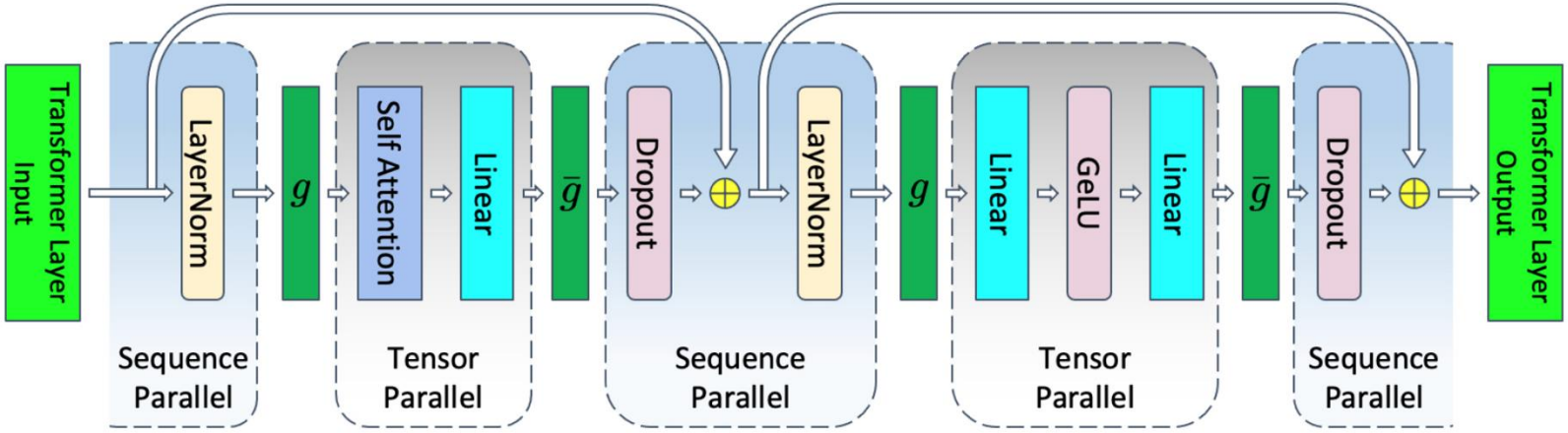
线性项来自若干[s,b,h]或[s,b,4h]张量：论文按模块计attention为11sbh+5abs^2，MLP为19sbh，两处LayerNorm共4sbh，所以11+19+4=34。进一步展开：MLP保存输入2sbh、GeLU输入8sbh、下投影输入8sbh及dropout mask sbh，共19sbh；attention线性项为共享QKV输入2sbh、Q/K共4sbh、V为2sbh、输出投影输入2sbh及dropout mask sbh，共11sbh。这里忽略相对h很小的归一化均值/方差等缓冲。二次项来自[b,a,s,s]注意力相关张量：两份16bit矩阵与1byte mask合成5abs^2。序列长度翻倍时线性部分翻倍，二次部分四倍，这解释了长上下文为何先撞激活墙。

TP后的剩余项

原页47写A_TP=sbh(10+24/t+5as/(ht))。其中24sbh与二次项能随头/MLP维度切到t卡；10sbh仍复制，包含LayerNorm相关4sbh、dropout 2sbh和attention/MLP输入4sbh。24/t并非“纯MLP项”，也包含attention里可分片的线性部分。t增大时复制的10sbh成为下界，说明仅TP不能使此账本完全除t。

本节参考（完整书目见讲义末尾）：[slides] · [activation]

Making memory truly linear – sequence parallel



Observation: all the 10sbh terms are pointwise ops over the sequence

… so split up the layer norm/dropout layers along the sequence axis.

- In the forward pass, 'g' is an all gather, '$\bar{g}$' is reduce-scatter
- In the backward pass, the two are reversed.

Making activation memory fully scale with more machines

Putting it together to get full linear scaling for memory.

Configuration	Activations Memory Per Transformer Layer
no parallelism	$sbh\left(34+5\frac{as}{h}\right)$
tensor parallel (baseline)	$sbh\left(10+\frac{24}{t}+5\frac{as}{ht}\right)$
tensor + sequence parallel	$sbh\left(\frac{34}{t}+5\frac{as}{ht}\right)$
tensor parallel + selective activation recomputation	$sbh\left(10+\frac{24}{t}\right)$
tensor parallel + sequence parallel + selective activation recomputation	$sbh(\frac{34}{t})$

第 8 讲 · 原页 48–49 · 中文讲解

SP与选择性重算：去掉复制项和二次保存项

沿序列切分哪些操作

LayerNorm对每个token的h维做归一化，dropout和残差也不需要别的token。因此可让每卡只处理s/t个token，并保留完整h维。这里“pointwise over sequence”是不同序列位置独立，不是LayerNorm对每个标量独立。进入按头/宽度分片的TP算子前，AllGather恢复所需完整序列；离开时用ReduceScatter归约并把输出按序列重新分片。反向两者互换。

为什么通常不增加主导通信量

原TP的AllReduce可拆成ReduceScatter+AllGather；SP将这两步分置在可按token独立计算的区段两端，让中间激活保持分片。在相同张量大小、分组和精度的理想带宽账本中，总载荷近似不变，但调用时序、延迟与重叠会改变。SP不等于把整段attention的KV上下文分片，后者要用CP等额外机制。

按表逐行理解

TP+SP把剩余10项也除t，变为(sbh/t)(34+5as/h)。选择性重算attention中占显存大的中间矩阵，用前向保存的小输入在反向重建，可去掉5abs^2的保存项：只有TP时为sbh(10+24/t)，TP+SP时为34sbh/t。它们限定论文中的保存策略；不是不可再压缩的绝对下界，全层checkpoint还能只存更少边界输入，但会增加更多重算。

编者数值检查

b=1、s=2048、h=4096、a=32、t=8时，sbh=8,388,608；无并行为912 MiB/层，TP为184 MiB，TP+SP为114 MiB，再做选择性重算为34 MiB。MiB按2^20字节换算。这些是该分析中保存的激活，不包含所有层、PP在途微批、参数及临时工作区；不能拿34 MiB直接宣称整模型显存。

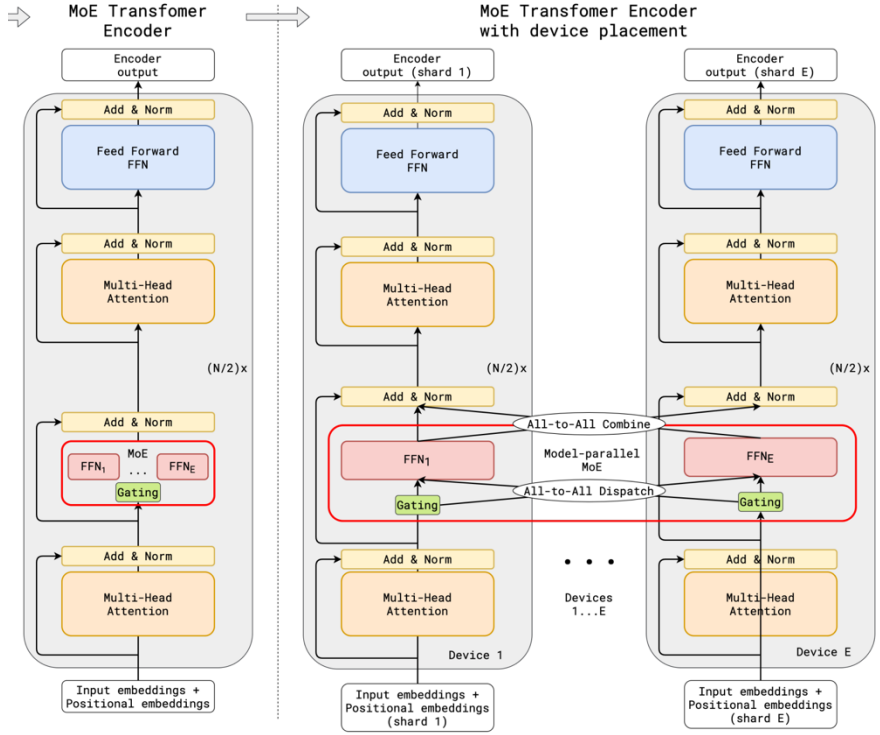
课堂补充 · 00:50:01–00:51:44；00:52:33–00:53:15

课堂把SP的按需汇集与FSDP类比：前者周转激活，后者周转权重。答疑也明确可以重算MLP，但要支付相应计算。因此表中34sbh/t是所选策略的结果，不应称作任何实现都不能突破的下界。

本节参考（完整书目见讲义末尾）：[slides] · [activation] · [captions]

Expert parallelism

Instead of splitting up the matmul, split up the experts and route activations



Why EP?

EP is *roughly* like TP in behavior for MLPs – high bandwidth, reduces activation

Guideline 4: Prefer EP over TP for Expert Layers

EP Advantages	Details
Better GEMM efficiency	Larger local matrix sizes improve GPU utilization
Lower communication	EP has less communication overhead than TP for MoE layers
Simpler computation graph	Easier to overlap communication with computation
Token permutation	When `EP = num_experts`, local token permutation is eliminated

Example: For Mixtral 8x7B, `EP8×TP1` outperforms `EP4×TP2`.

But, splitting matmuls can reduce efficiency vs routing activations

第 8 讲 · 原页 50–51 · 中文讲解

EP：把token送到专家，而不是把每个专家都切碎

从MoE图读通信

每个token的router选择少数专家，专家权重分布在不同设备。先按目标专家整理token并dispatch，目标卡执行本地专家MLP，再把输出送回原token位置并按路由权重合并。这里AllToAll的语义是每个rank给其他rank不同数据块，与AllGather让所有人拿同一完整集合不同。动态图里红色FFN框表示被路由访问的专家；attention仍是常规稠密部分。

编者两卡例子

两卡各持2个专家，4个token做top-1路由，若其中3个都选rank1的同一专家，则rank1会更忙；专家权重分得平均并不保证token负载平均。top-k还可能把同一token送给k个专家，流量、padding、容量因子与token丢弃策略都影响性能。局部可用token太少时，专家GEMM同样变小，硬件利用率下降。

EP与TP的选择边界

TP把一个专家矩阵切薄，需多卡合作并归约；EP尽量保留较完整的本地专家矩阵，以路由通信换取较大的GEMM。原页51中的优势是条件性的：网络、专家数、top-k、负载均衡、token数和内核实现都会改变结果。EP仅直接按组数缩小专家参数，稠密层仍可能复制；激活变化还取决于路由，不能承诺所有激活都线性缩小。

本节参考（完整书目见讲义末尾）：[slides] · [moe]

Complexity – combining EP and others

Naively – we can compose EP and all other strategies..

But there are important notes – DP usually shares replicas with EP splits (so EP<DP)

DP and TP can interact badly to lower utilization

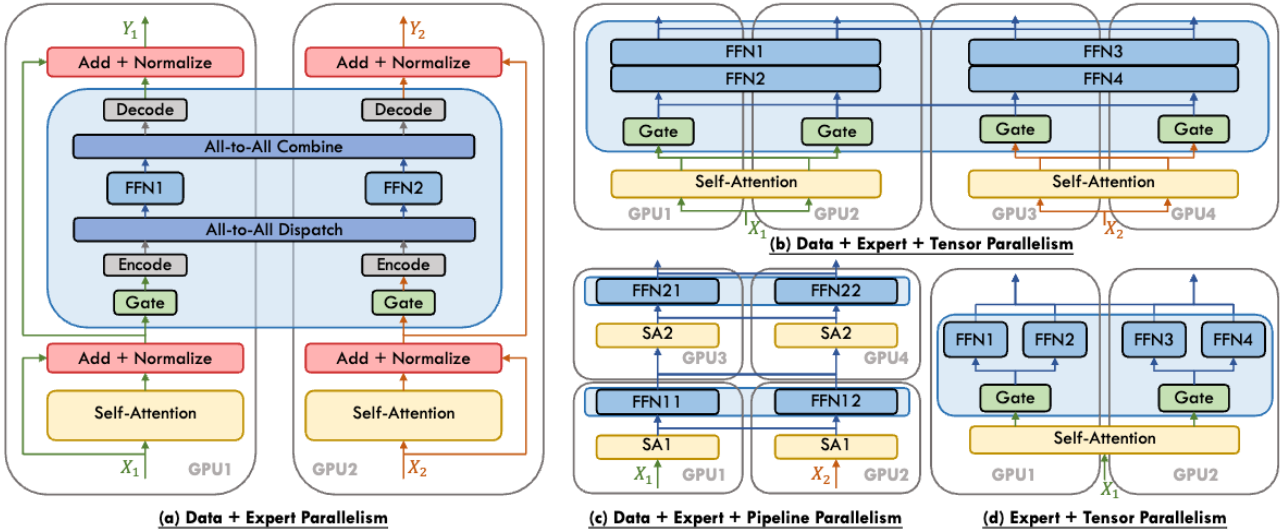


Fig. 8. Schematic depiction of diverse parallel strategies for MoE. For clarity and conciseness, this illustration omits some All-to-All, All-Reduce, Point-to-Point communication within parallelism, and Normalization, Encode, Decode, Gate in subfigures (b), (c), and (d).

Complexity: Decoupling attention / expert parallelism

Recall: MoEs apply to the MLPs, not the attention (with some exotic exceptions)

This can lead to some imbalance between attention and MLPs for parallelism

- High TP – useful for attention parallelization (since we cant use EP)
- Low TP – useful for MLP parallelization (we'd rather EP than TP)

What to do? More flexible parallelism strategies (i.e. in Megatron)

MoE Parallel Folding is Megatron Core's solution that **decouples attention and MoE parallelism**:

Parallelism Group	Attention Layers	MoE Layers
Dimensions	TP × CP × DP × PP	ETP × EP × EDP × PP

Separates out TP/CP/DP (for attention) and (ETP/EP/EDP) for MLPs

https://docs.nvidia.com/megatron-core/developer-guide/latest/user-guide/features/moe.html

第 8 讲 · 原页 52–53 · 中文讲解

EP与其他并行的组合：先画通信组再乘数字

为什么EP常复用DP设备

在稠密层，不同卡可处理不同样本并复制同一权重；进入MoE时，同一批卡改为各持部分专家，通过token路由共同处理样本。稠密权重的DP组与专家权重的复制组因而不相同。原页“EP<DP”是在特定分组惯例下的经验写法，更一般要检查EP是否可整除相关组以及哪些轴复用，而不是把严格小于当算法定律。

编者8卡分组例子

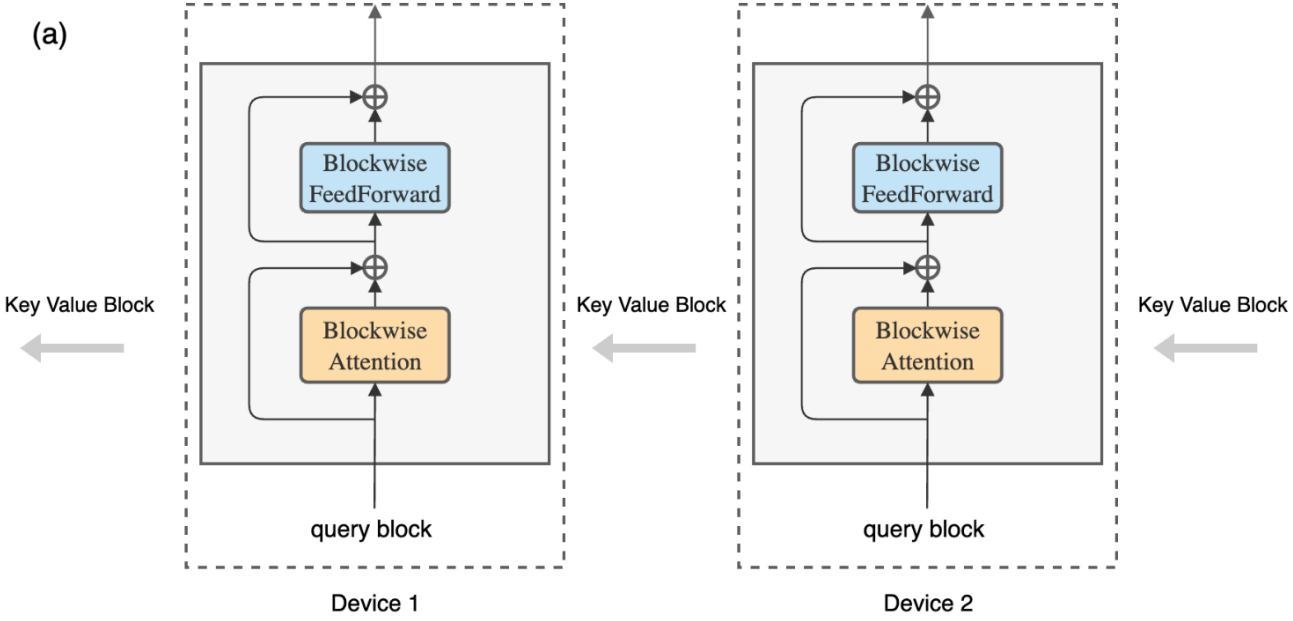
设8卡没有TP和PP，稠密层有8路DP；MoE层把它们分为两个4卡EP组，每组容纳完整专家集合。此时EP=4、专家数据并行EDP=2，但总卡数仍是8，而不是DP8×EP4=32。两组里对应专家的权重梯度需要跨副本同步；同一EP组内的不同专家并非同一参数的副本，不能用普通DP把它们互相平均。

第53页的folding表

Attention一侧的设备划分写TP×CP×DP×PP，MoE一侧写ETP×EP×EDP×PP，两边描述同一批物理设备，乘积应一致。ETP是专家内部张量并行，EDP是专家副本并行。让attention有较大TP、专家有较小ETP与较大EP，可以适应两类算子的不同瓶颈；代价是布局转换、路由与实现复杂度。“可组合”不意味着任意并行度都能随意相乘。

本节参考（完整书目见讲义末尾）：[slides] · [moe]

Other parallelism strategies



Context parallel / Ring attention (split activations across GPUs in a long sequence)

第 8 讲 · 原页 54–54 · 中文讲解

CP与Ring Attention：分片完整上下文的工作

与SP的差别

本讲SP主要切分LayerNorm/dropout等可按token独立的区段；CP把一个长序列的token及attention工作也分给多卡。卡i拥有一段Q，但精确全注意力仍需对应全部K、V。仅在本卡片段内做softmax会改变模型，因为漏掉了跨片段注意力。

按图理解环形传递

图中query block保留在本地，Key Value block沿设备环传递。每张卡逐次用自己的Q与收到的K/V块计算局部scores，并维护全局归一化所需统计，直到覆盖需要的上下文。编者说明：合并不同块时要使用在线softmax的共同最大值和归一化分母；不能把各块单独softmax的输出简单相加。反向同样要处理跨卡贡献。

收益与限制

CP可以降低每卡保存的序列激活并分摊attention计算，但并未把全注意力的总s^2计算魔法般变成s。通信是否能与分块计算重叠、因果mask是否让某些卡更忙、块大小是否高效，都影响吞吐。长上下文阶段常把原来的DP设备转给CP，而不是单纯加大全局batch。

本节参考（完整书目见讲义末尾）：[slides] · [ring]

Recap: LLM parallelism table

Method	Comm/Sync	Param memory per rank	Activation / KV memory per rank	Main bandwidth cost	Scales global batch?	Easy to use?
DDP / ZeRO-1	gradient all-reduce per step	**No param scaling** (ZeRO-1 only opt state)	**None**	**Gradient traffic ~ O(params)**	**Yes, linear in DP**	**Very**
FSDP / ZeRO-3	Gradient all reduce – can be overlapped	**~1/DP** for params/grads/optimizer state	**None**	**Param traffic ~ O(params)**, higher than DDP, overlap	**Yes, linear in DP**	**Moderate**
Pipeline parallel	Activation between layers, **pipeline bubbles**	**~1/PP**	Depends on pipeline buffers	**Activation traffic between stages**	**No**, but needs microbatches	**Hard**
Tensor parallel	**Blocking activation communication**	**~1/TP** for TP-sharded weights	**~1/TP** for relevant matmul activations w/ SP	**Activation-sized collectives every block**	**No**	**Hard**
Sequence / Context parallel	per-layer sequence-shard exchange	**None**	**~1/SP or 1/CP** on sequence-side activations / KV	**Activation / KV communication**	**No**	**Hard**
Expert parallel (MoE)	**token dispatch - all-to-all per MoE**	**~1/EP for expert weights only**	**None**	**Token-routing all-to-all**	**No**, but needs enough tokens per expert	**Hard**

第 8 讲 · 原页 55–55 · 中文讲解

用总表检查每种并行到底解决了什么

对原表作精确阅读

DDP复制参数并同步梯度；ZeRO-1只分片优化器状态；FSDP/ZeRO-3主路径是参数AllGather与梯度ReduceScatter，原表笼统写gradient all-reduce不能替代这一生命周期。PP按层持有参数并传边界激活；TP分割相关权重并频繁交换激活；SP/CP主要减激活存储；EP分割专家权重并路由token。

表中的“None”和“linear”都有条件

DP的激活None表示固定每副本batch时没有专门分片；固定全局batch而增加DP，局部激活仍可能变小。EP的激活None表示没有普适的线性缩放保证，并不否认某些布局减少局部保存量。PP的参数近1/p假设层/参数均衡，TP近1/t仅限被切分的权重。KV列需要按推理场景单独解释，训练状态与推理缓存不能互相代算。

编者诊断练习

若模型状态已装下但梯度同步慢，增加DP不一定解决问题，应检查局部batch、分桶和重叠；若只在长序列OOM，继续切优化器可能作用有限，应查激活、attention保存、SP/CP与重算；若PP空闲长而网络不忙，先看微批数和阶段负载。这个表应作为瓶颈排查入口，而非只看“Easy to use”一列选方案。

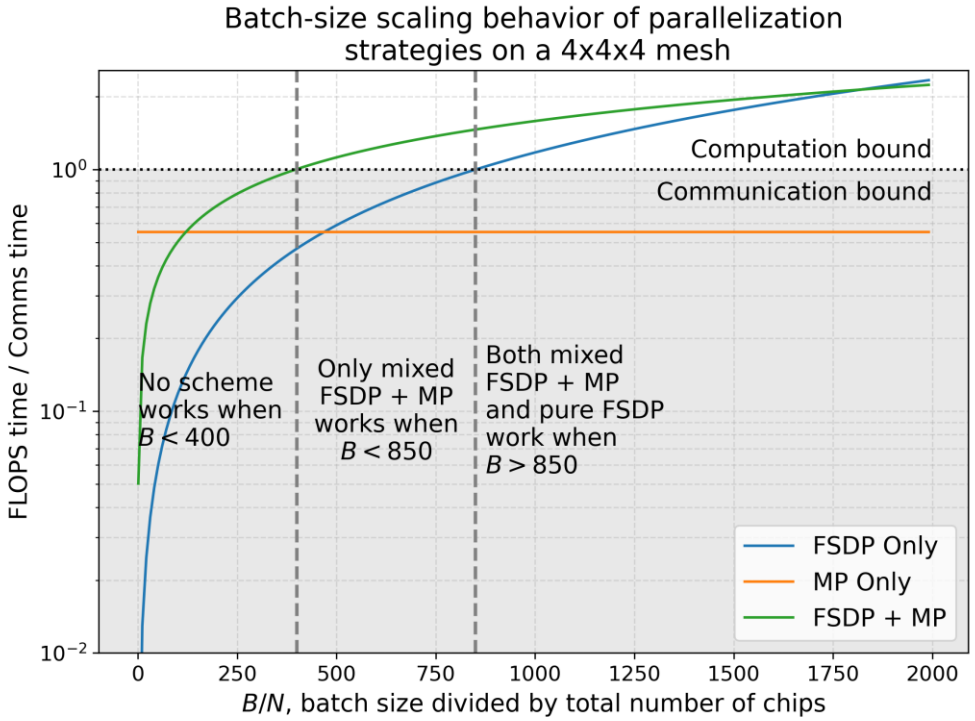
本节参考（完整书目见讲义末尾）：[slides]

Model vs Tensor parallel (TPU book)

Key quantity

Global batch size (divided by GPU)

Strategy	Compute per layer (ignoring gating einsum)	Comms per layer (bytes, forward + backward pass)
DP	$4BDF/X + 8BDF/X$	$0 + 8DF$
FSDP	$4BDF/X + 8BDF/X$	$4DF + 8DF$
MP	$4BDF/Y + 8BDF/Y$	$4BD + 4BD$
FSDP + MP	$4BDF/(XY) + 8BDF/(XY)$	$(4BD/X + 4DF/Y) + (8BD/X + 8DF/Y)$



第 8 讲 · 原页 56–56 · 中文讲解

以每卡batch理解通信与计算的交叉点

读图轴与曲线

横轴B/N是全局batch除总芯片数；此图的B按token计数，等长序列时等于本文全局序列样本数乘s。因此400、850是每芯片token量级，不是对应条数的序列。纵轴是计算时间与通信时间的相对关系，用以判断计算或通信主导。图里的400、850等交叉点属于该TPU网格、矩阵规模和带宽假设下的算例，不是GPU训练的通用阈值。下方表用D、F等局部符号表示层宽与MLP宽，不能把其D误读为本讲统一记号里的DP组大小d。

编者用缩放关系理解曲线

以线性层XW为例，W含P层参数，某卡处理T个token。前反向计算量与T×P层成正比。FSDP汇集权重的字节数主要随P层变化，T增大时每次搬权重可服务更多token，因此通信更容易摊薄。TP交换的激活则也随T增长，计算/通信的比值更受隐藏维与分片度支配。两者混合能分别减少某些通信压力，但引入分组与更多布局。

如何迁移结论

使用自己的矩阵形状、有效GEMM吞吐、集合通信时间和高速域重新估算T计算/T通信，再看峰值显存是否允许该局部batch。原图里混合策略更早进入计算主导区间的现象，可以指导搜索方向，却不能推出对任意小batch都有解。

本节参考（完整书目见讲义末尾）：[slides] · [scaling-book]

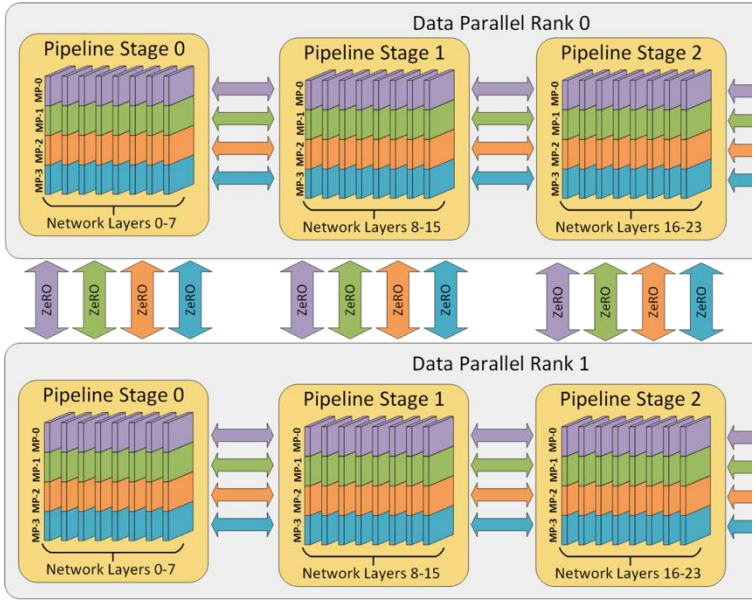
‘3D (4D) parallelism’ – putting it all together

Simple rules of thumb from the literature.

1. Until your model fits in memory..
   - Tensor / expert parallel up to GPUs / machine
   - Pipeline parallel across machines
   - (Or use Zero-3, depending on BW)

2. Then until you run out of GPUs
   - Scale the rest of the way with data parallel

If your batch size is small.. gradient accumulate to trade higher batch sizes for better communication efficiency.



Example from Metagron's current recommendations

The "**basics**"

Guideline 1: Minimize Model Parallelism, Maximize Data Parallelism

Aspect	Recommendation
Goal	Keep TP/EP/PP as small as possible while avoiding OOM
Why	Model parallelism introduces communication overhead that hurts performance
How	Use distributed optimizer (`--use-distributed-optimizer`) to shard optimizer states across DP ranks, freeing memory for larger DP size

Guideline 2: Keep EP and TP Communication Within NVLink Domain

Aspect	Recommendation
Goal	Ensure EP×TP fits within a single node (typically 8 GPUs)
Why	EP and TP are communication-intensive; NVLink provides much higher bandwidth than cross-node interconnects
Scaling	When scaling beyond one node, prefer PP over expanding TP/EP across nodes

Note: For very large MoE models like DeepSeek-V3, the EP communication may exceed the NVLink bandwidth. In this case, consider using 1F1B A2A Overlap to overlap the EP communication.

Guideline 3: Use Pipeline Parallelism (PP) for Multi-Node Scaling

Aspect	Recommendation
Goal	Use PP to distribute layers across nodes while keeping EP×TP within NVLink
VPP	Enable Virtual Pipeline Parallelism to reduce pipeline bubbles when `PP ≥ 2`
Config	Set `--num-layers-per-virtual-pipeline-stage` to control VPP size

Plus more advanced "**extras**"

Guideline 4: Prefer EP over TP for Expert Layers

EP Advantages	Details
Better GEMM efficiency	Larger local matrix sizes improve GPU utilization
Lower communication	EP has less communication overhead than TP for MoE layers
Simpler computation graph	Easier to overlap communication with computation
Token permutation	When `EP = num_experts`, local token permutation is eliminated

Example: For Mixtral 8x7B, `EP8×TP1` outperforms `EP4×TP2`.

Guideline 5: Enable Context Parallelism (CP) for Long Sequences

Aspect	Recommendation
When to use	Sequence length ≥ 8K tokens
Key factor	CP efficiency depends on overlapping communication with computation
Config	Set `--context-parallel-size` to partition sequences across GPUs

https://docs.nvidia.com/megatron-core/developer-guide/latest/user-guide/features/moe.html

第 8 讲 · 原页 57–58 · 中文讲解

设计一个并行布局：先容纳，再把剩余卡用于数据

原图三维网格的含义

第57页水平方向是PP阶段，每个阶段内部的小块是TP分片；上下两套网络是DP副本。普通独立三维布局N=d×t×p。TP优先留在高速域，PP可跨较慢链接；若跨域带宽与预取足够，FSDP也可能合适。第58页的建议是先尽量降低必要模型并行度，再把多出的设备用于DP，同时用EP、CP处理专家和长上下文的特殊瓶颈。

编者64卡方案

假设8台机器各8卡，需t=8、p=4才能装下模型，则d=64/(8×4)=2。取b=1、m=16，B=2×1×16=32个序列；理想PP利用率16/(16+4−1)≈84.2%。TP只在同节点通信，PP阶段之间跨节点，同一阶段同一TP坐标的两份副本构成DP组。这个练习只保证算术和组结构自洽，实际还要验证层均衡、峰值和链路布局。

梯度累积到底改变了什么

增大m而保持b可增大全局有效batch，并减少每个样本分摊的某些梯度同步开销，但不能消除每微批TP/PP通信。若全局B必须固定，m的增加要由b或d的减少抵消，不是免费得到更多样本。FSDP累积期间是否每微批汇集/归约由配置决定。MoE建议把EP放高速域与跨节点大EP案例并不矛盾：后者为容量与专家规模接受了更复杂的路由优化。

本节参考（完整书目见讲义末尾）：[slides] · [moe]

Scaling strategies from Narayanan 2021

Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM

Deepak Narayanan‡*, Mohammad Shoeybi†, Jared Casper†, Patrick LeGresley†,
Mostofa Patwary†, Vijay Korthikanti†, Dmitri Vainbrand†, Prethvi Kashinkunti†,
Julie Bernauer†, Bryan Catanzaro†, Amar Phanishayee*, Matei Zaharia‡
†*NVIDIA* ‡*Stanford University* **Microsoft Research*

Number of parameters (billion)	Attention heads	Hidden size	Number of layers	Tensor model-parallel size	Pipeline model-parallel size	Number of GPUs	Batch size	Achieved teraFlOP/s per GPU	Percentage of theoretical peak FLOP/s	Achieved aggregate petaFLOP/s	DP size
1.7	24	2304	24	1	1	32	512	137	44%	4.4	32
3.6	32	3072	30	2	1	64	512	138	44%	8.8	32
7.5	32	4096	36	4	1	128	512	142	46%	18.2	32
18.4	48	6144	40	8	1	256	1024	135	43%	34.6	32
39.1	64	8192	48	8	2	512	1536	138	44%	70.8	32
76.1	80	10240	60	8	4	1024	1792	140	45%	143.8	32
145.6	96	12288	80	8	8	1536	2304	148	47%	227.1	24
310.1	128	16384	96	8	16	1920	2160	155	50%	297.4	15
529.6	128	20480	105	8	35	2520	2520	163	52%	410.2	9
1008.0	160	25600	128	8	64	3072	3072	163	52%	502.0	6

Notes

- Tensor parallel first up to 8, then caps out at 8.
- Pipeline parallel goes up to make the model fit.
- Dara parallel gradually decreases with scale, with the largest model having DP=6

Careful ‘3D’ parallelism gives linear gains

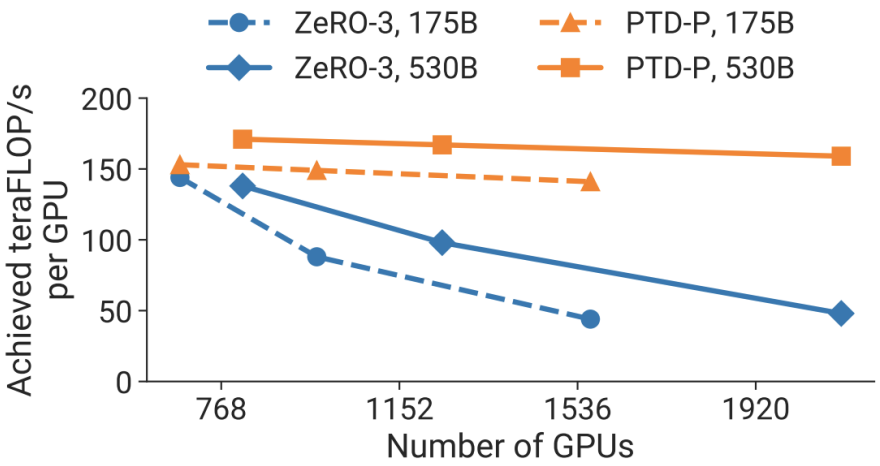


Figure 10: Throughput per GPU of PTD-P and ZeRO-3 for two different GPT models (the 175B GPT-3 model is shown with dotted lines, and the 530B model is shown with solid lines). Global batch sizes are fixed and ZeRO-3 is used without any model parallelism.

More GPUS, same, flat utilization!

第 8 讲 · 原页 59–60 · 中文讲解

历史规模实验：检查乘积与固定条件

读第**59**页表的几列

参数数、层数和隐藏维共同决定计算与容量；TP度先增长到8，再固定在该系统的机内范围，PP随模型变深/变大而增大。表里总GPU数不是一味用来加DP，而是重新分配给模型并行。编者验算：145.6B行1536卡、TP8、PP8，所以DP=1536/64=24；约1T行3072卡、TP8、PP64，所以DP=6。

每卡吞吐与总体吞吐

第60页纵轴为每卡 achieved teraFLOP/s，横轴为GPU数。若每卡有效吞吐近似不变，总有效吞吐就近似随卡数线性增加。图中模型为175B/530B、全局batch固定，橙色组合策略比蓝色纯ZeRO-3更能保持每卡效率；不能把该2021系统测量直接当成所有新版FSDP的结论，也不能把不同模型的曲线当相同工作量。

编者复核习惯

复现扩展结论时应固定模型、序列长、全局batch、精度和计算量口径，注明是否包括重算FLOPs。若这些条件随规模改变，需要区分强扩展与弱扩展。表中“利用率”应看有效训练计算还是硬件实际执行计算的分母定义，避免把重算越多误认为模型训练效率越高。

本节参考（完整书目见讲义末尾）：[slides] · [megatron]

Tensor parallel = 8 is often optimal

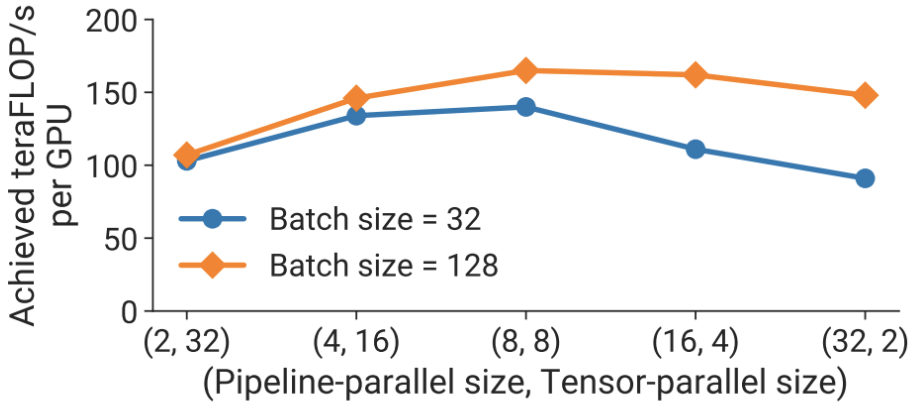
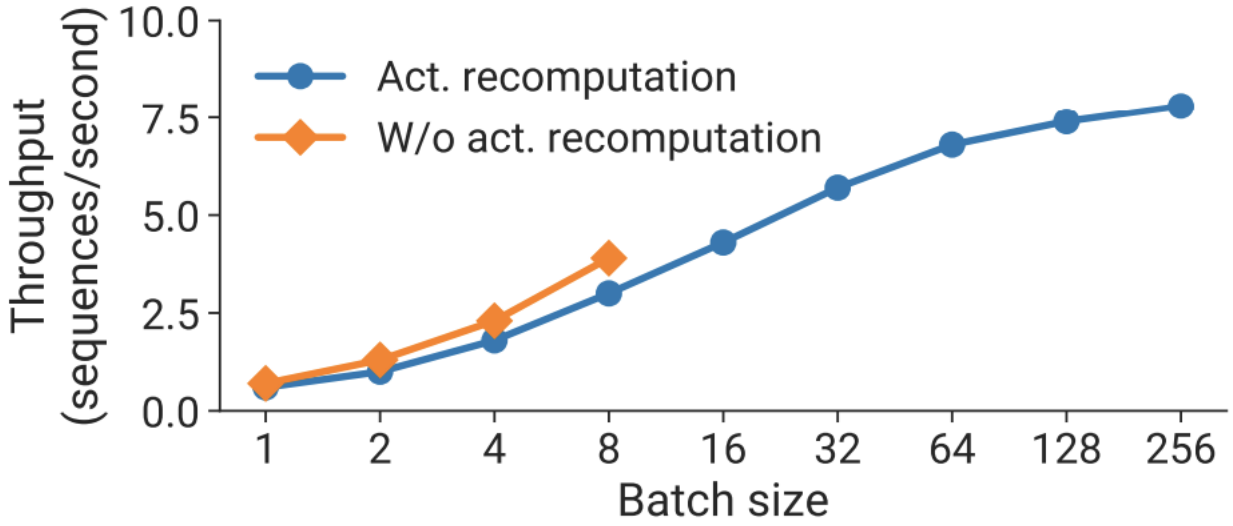


Figure 13: Throughput per GPU of various parallel configurations that combine pipeline and tensor model parallelism using a GPT model with 162.2 billion parameters and 64 A100 GPUs.

When parallelizing across 64 machines – it's best to use a 8 x 8 configuration.

Activation recomputation can pay for itself (via memory)



Activation recomputation enables larger batches, improving throughput (t=8, p=16)

第 8 讲 · 原页 61–62 · 中文讲解

TP/PP取舍与“多算反而更快”的条件

第61页横轴是一对数

图中横轴为(PP,TP)，从(2,32)到(32,2)，乘积固定64；图注明确写64张A100 GPU，原页正文的“64 machines”应作为措辞问题保留并在此澄清。中间(8,8)较好，是过大TP的跨域通信与过大PP的气泡之间的折中，不是8的数学最优性。蓝/橙曲线还显示不同batch会改变折中。

第62页：比较可行的运行点

橙色不重算曲线在能跑的相同小batch下通常更快；但它更早触及显存容量，无法继续放大batch。蓝色重算增加计算，却释放显存，使更大batch可运行、提高GEMM效率并减小相对气泡，最终吞吐可能更高。不能把“重算收益”解释为重算本身减少FLOPs。

编者判断步骤

先比较同batch、同布局下重算的额外时间，再检查它是否换来更大的可行局部batch或更好的并行布局；最终比较在目标全局batch与优化约束下的tokens/sec。若固定全局batch不允许继续增大、或本来已计算饱和，重算可能只会变慢。原图给的是存在这种系统收益的实例。

本节参考（完整书目见讲义末尾）：[slides] · [megatron] · [activation]

Recent LMs – what do they do?

3.1 Distributed Training Framework

We train our models using the *ZeRO* optimizer strategy (Rajbhandari et al., 2019) via PyTorch's FSDP framework (Zhao et al., 2023), which reduces memory consumption by sharding the model weights and their corresponding optimizer state across GPUs. At the 7B scale, this enables training with a micro-batch size of 4096 tokens per GPU on our hardware (see Section 3.4). For OLMo-1B and -7B models, we use a constant global batch size of approximately 4M tokens (2048 instances, each with a sequence length of 2048 tokens). For OLMo-65B model (currently training), we use a batch size warmup that starts at approximately 2M tokens (1024 instances), then doubles every 100B tokens until reaching approximately 16M tokens (8192 instances).

To improve throughput, we employ mixed-precision training (Micikevicius et al., 2017) through FSDP's built-in settings and PyTorch's `amp` module. The latter ensures that certain operations like the softmax always run in full precision to improve stability, while all other operations run in half-precision with the `bfloat16` format. Under our specific settings, the sharded model weights and optimizer state local to each GPU are kept in full precision. The weights within each transformer block are only cast to `bfloat16` when the full-sized parameters are materialized on each GPU during the forward and backward passes. Gradients are reduced across GPUs in full precision.

Dolma – 7B model, FDSP (probably fits intra-node)

第 8 讲 · 原页 63–63 · 中文讲解

OLMo案例：FSDP也是完整训练方案

原稿名称澄清

本页标题下写Dolma – 7B model，插图讨论的是OLMo模型训练；Dolma是其数据集名称。字幕01:12:46–01:13:03中教师当场把Dolma更正为OLMo。这一处应沿用课堂纠正来理解，原英文页仍完整保留。

从截图读出混合精度与分片

插图描述通过PyTorch FSDP分片模型权重和优化器状态，并在Transformer块执行时汇集完整参数；还区分保存权重、算子精度和梯度归约精度。它说明“BF16训练”不代表所有对象都BF16，也说明中等规模模型可以用FSDP作为主要分布式方案。原页“probably fits intra-node”是推测，不能据此认定整个训练只使用单节点。更大训练集吞吐仍可能驱动跨节点DP/FSDP扩展。

本节参考（完整书目见讲义末尾）：[slides] · [captions]

DeepSeek

2.4. Infrastructures

We use an efficient and light-weight training framework named HAI-LLM (High-flyer, 2023) to train and evaluate large language models. Data parallelism, tensor parallelism, sequence parallelism, and 1F1B pipeline parallelism are integrated into this framework as done in Megatron (Korthikanti et al., 2023; Narayanan et al., 2021; Shoeybi et al., 2019). We also leverage the flash attention (Dao, 2023; Dao et al., 2022) technique to improve hardware utilization. ZeRO-1 (Rajbhandari et al., 2020) is exploited to partition optimizer states over data parallel ranks. Efforts are also made to overlap computation and communication to minimize additional waiting overhead, including the backward procedure of the last micro-batch and reduce-scatter operation in ZeRO-1, and GEMM computation and all-gather/reduce-scatter in sequence parallel. Some layers/operators are fused to speed up training, including LayerNorm, GEMM whenever possible, and Adam updates. To improve model training stability, we train the model in bf16 precision but accumulate gradients in fp32 precision. In-place cross-entropy is performed to reduce GPU memory consumption, i.e.: we convert bf16 logits to fp32 precision on the fly in the cross-entropy CUDA kernel (instead of converting it beforehand in HBM), calculate the corresponding bf16 gradient, and overwrite logits with its gradient.

DeepSeek – ZeRO stage 1 with Tensor, Sequence, and Pipeline parallel

V3 – PP (16), EP (64-way, 8 nodes), ZeRO stage 1

EP uses 1F1B A2A Overlap

Yi

Performance and Cost Efficiency

Memory and *communication* restrictions are the two major technical challenges of large scale model training requiring integrated solutions beyond adding more GPUs. We use and improve upon the following techniques to tackle the memory and communication restrictions: (1) ZeRO-1 [60] to remove the memory consumption by partitioning optimizer states cross data-parallel processes; (2) tensor parallel combined with pipeline parallel [70] within each compute node to avoid inter-node communication bottleneck, and the 3D parallel strategy is well designed and optimized to avoid using activation checkpointing and minimize the pipeline bubbles; (3) kernel fusion techniques like flash attention [15] [14] and JIT kernels to reduce redundant global memory access and consumption; (4) topology-aware resource allocation (ranking strategy) to minimize the communication across different layers of switches, which is the limitation of a typical fat-tree-topology.

Yi - ZeRO stage 1 + Tensor + Pipeline parallel

Yi-lightning (2025) – Tensor replaced by Expert parallelism

第 8 讲 · 原页 64–65 · 中文讲解

DeepSeek与Yi：状态分片可叠加模型并行

先分清模型版本

第64页截图描述早期DeepSeek的DP、TP、SP和1F1B PP，并用ZeRO-1分片优化器状态；下方另列DeepSeek-V3的PP16、EP64和ZeRO-1。两段是不同模型/配置，不能把早期段落里所有TP/SP细节无条件接到V3上。V3技术报告说明其PP、跨节点EP及通信计算重叠；原页的简短1F1B A2A表述不足以完整定义其DualPipe调度。

编者解释：ZeRO-1不是与TP二选一

模型先被TP/PP或EP分到各设备后，同一参数分片还可在多个数据副本上复制；ZeRO-1针对这些副本分配主权重与优化器状态的更新责任。它不要求所有卡保存原始完整模型。网络快且状态分片不必非常激进时，这种组合能减少参数反复汇集，代价是仍有部分复制和更复杂的通信组。

Yi页的可迁移内容

第65页早期Yi案例也把ZeRO-1、TP与PP组合，并强调拓扑感知分配和减少气泡。下方Yi-lightning是另一代MoE案例，原页称用EP替代TP，应理解为专家层布局的变化，不能推成“以后所有层都不需要TP”。读模型报告时要分别查稠密attention、专家层、数据副本以及训练阶段，而不是给一个模型只贴一种并行标签。

本节参考（完整书目见讲义末尾）：[slides] · [deepseek]

Llama3 405B

GPUs	TP	CP	PP	DP	Seq. Len.	Batch size/DP	Tokens/Batch	TFLOPs/GPU	BF16 MFU
8,192	8	1	16	64	8,192	32	16M	430	43%
16,384	8	1	16	128	8,192	16	16M	400	41%
16,384	8	16	16	8	131,072	16	16M	380	38%

(Stage 1, small bsz training, Stage 2 pretraining, Stage 3 long-context)

Network-aware parallelism configuration. The order of parallelism dimensions, [TP, CP, PP, DP], is optimized for network communication. The innermost parallelism requires the highest network bandwidth and lowest latency, and hence is usually constrained to within the same server. The outermost parallelism may spread across a multi-hop network and should tolerate higher network latency. Therefore, based on the requirements for network bandwidth and latency, we place parallelism dimensions in the order of [TP, CP, PP, DP]. DP (*i.e.*, FSDP) is the outermost parallelism because it can tolerate longer network latency by asynchronously prefetching sharded model weights and reducing gradients. Identifying the optimal parallelism configuration with minimal communication overhead while avoiding GPU memory overflow is challenging. We develop a memory consumption estimator and a performance-projection tool which helped us explore various parallelism configurations and project overall training performance and identify memory gaps effectively.

Llama 3 405B

Side note – Lots of GPU failures at this scale!

Component	Category	Interruption Count	% of Interruptions
Faulty GPU	GPU	148	30.1%
GPU HBM3 Memory	GPU	72	17.2%
Software Bug	Dependency	54	12.9%
Network Switch/Cable	Network	35	8.4%
Host Maintenance	Unplanned Maintenance	32	7.6%
GPU SRAM Memory	GPU	19	4.5%
GPU System Processor	GPU	17	4.1%
NIC	Host	7	1.7%
NCCL Watchdog Timeouts	Unknown	7	1.7%
Silent Data Corruption	GPU	6	1.4%
GPU Thermal Interface + Sensor	GPU	6	1.4%
SSD	Host	3	0.7%
Power Supply	Host	3	0.7%
Server Chassis	Host	2	0.5%
IO Expansion Board	Host	2	0.5%
Dependency	Dependency	2	0.5%
CPU	Host	2	0.5%
System Memory	Host	2	0.5%

Table 5 Root-cause categorization of unexpected interruptions during a 54-day period of Llama 3 405B pre-training. About 78% of unexpected interruptions were attributed to confirmed or suspected hardware issues.

第 8 讲 · 原页 66–67 · 中文讲解

Llama 3：并行布局随训练阶段改变，可靠性也计入效率

按原表逐行验算

表头次序是GPUs、TP、CP、PP、DP。第一行8192=8×1×16×64；常规预训练行16384=8×1×16×128；长上下文行16384=8×16×16×8。长上下文把CP从1升到16、DP从128降到8，总卡数不变。单个序列由更多卡合作，正好应对更大的激活与attention工作。第72页只列常规预训练行，不能据此说长上下文也CP=1。

batch数也要乘对轴

常规行每个DP副本每步16个序列、序列8192，故128×16×8192=16,777,216 tokens，约16M；32个序列属于前一行DP=64的较小规模配置；长上下文行8×16×131072=16,777,216 tokens，约16M。这里batch size/DP是每个数据副本整步的样本数，不是TP每卡再独立取的batch。TP/CP/PP协作同一批样本，不能再乘入token总数。

第67页故障表的边界

表统计54天窗口的意外中断分类，Faulty GPU一项148次；GPU HBM等另列，因此148不是全部GPU相关故障，也不是所有训练中断总数。硬件、网络、软件和维护都会打断大集群。恢复要保存足以继续训练的模型/优化器/步数等状态，并考虑分片检查点的重组；稳态吞吐很高而频繁丢失进度，最终有效训练效率仍会差。

课堂补充 · 01:14:50–01:16:24

课堂特别强调Llama报告分别列出多个训练阶段，长上下文提高CP并降低DP；随后借故障统计指出，大规模训练还包含分布式可靠性问题。本文按原论文表的统计范围解释148，避免把课堂简略措辞扩大成所有中断。

本节参考（完整书目见讲义末尾）：[slides] · [llama] · [captions]

Gemma 2

For 2, 9, 27B models

ZeRO-3, MP (=TP+SP), DP

3.3. Compute Infrastructure

We train our models with TPUv4, TPUv5e, and TPUv5p as outlined in Table 3. For the 2B model, we train on a 2x16x16 configuration of TPUv5e, totaling 512 chips, with 512-way data replication and 1-way model sharding. For the 9B model, we train on an 8x16x32 configuration of TPUv4, totaling 4096 chips, with 1024-way data replication and 4-way model sharding. For the 27B model, we train on an 8x24x32 configuration of TPUv5p, totaling 6144 chips, with 768-way data replication and 8-way model sharding.

The optimizer state is further sharded using techniques similar to ZeRO-3 (Ren et al., 2021). For scales beyond a single pod, we perform a data-replica reduction over the data center network, using the Pathways approach of Barham et al. (2022). We also use the 'single controller' programming paradigm of Jax (Roberts et al., 2023) and Pathways (Barham et al., 2022). As in Gemma 1, we use the GSPMD partitioner (Xu et al., 2021) for training step computation and the MegaScale XLA compiler (XLA, 2019).

第 8 讲 · 原页 68–68 · 中文讲解

Gemma 2：不同硬件仍在解同样的切分问题

读配置段落

本页列2B、9B、27B模型，并以ZeRO-3、MP与DP概括TPU训练策略；这里MP在课件语境指TP/SP式模型分片，不能看到MP就自动理解为PP。截图中不同模型使用不同的data replication与model sharding比例，说明参数规模与每设备内存共同决定切分程度。

编者验算与边界

27B示例的8×24×32共6144个芯片，对应768路数据复制×8路模型分片=6144，正好解释第72页Gemma行的数字。此处应沿用原报告的芯片单位，不把它换成GPU台数。实现依靠TPU编译器/SPMD分片等机制，ZeRO-3是对状态布局思想的描述，并不等于运行PyTorch FSDP。其优化器额外分片方式仍要结合原实现，不能仅凭这一行反推通信次数。

本节参考（完整书目见讲义末尾）：[slides]

Mixtral 8x22B

From megatron docs –

TP / PP / CP / EP (4/4/1/8) – DP likely 2 (to add up to 256GPUs)

Mixture-of-Experts Models

Model	Size	GPUs	TP	PP	CP	EP	Configuration Notes
Mixtral	8x7B	64	1	4	1	8	EP=8 for 8 experts
Mixtral	8x22B	256	4	4	1	8	TP+PP+EP for large MoE
DeepSeek-V3	671B	1024	2	16	1	64	Massive MoE with 256 experts

Nemotron 3 Super 120B-A12B

From the long-context extension section

Similar to Nemotron 3 Nano, we added a long-context phase (LC-Phase) at the end of pretraining. In the LC-Phase, we performed continuous pretraining (CPT) to equip the base model with long-context ability. We used a constant learning rate of $4.5 * 10^{-6}$ and global batch size of 16. We used 64-way context parallelism, 2-way tensor parallelism, and 64-way expert parallelism to train on GB200 GPUs. We reused the long-context document QA dataset from Nemotron 2 & 3 Nano. We allocated the document QA data to 20% in the Phase LC data blend, with the remaining 80% being downscaled Phase 2 data. We initially performed CPT on 1,048,576 (1m) context length. Such stage lasted for 34 billion tokens. Following that we added another stage to alternatingly train on both 1m and 4k sequences in order to mitigate the minor impact we observed on the math-related benchmarks. The second stage lasted for 17 billion tokens.

TP / PP / CP / EP (2/0/64/64)

第 8 讲 · 原页 69–70 · 中文讲解

MoE与长上下文案例：不要把所有并行度相乘

第69页：推荐配置不等于原训练披露

原页给Megatron文档的Mixtral 8×22B配置TP4、PP4、CP1、EP8、256卡，并写DP likely2。这个2来自把EP也当独立轴后做256/(4×4×1×8)，但常见实现中EP复用稠密DP轴，因此它只能作为原页推测保留，不能当已证实的DP。若该布局采用稠密N=TP×PP×CP×DP，则稠密DP=16；若EP8在此轴内分组，专家副本度可能为2。具体需要通信组定义才能确定。

第70页：CP与EP服务不同层

Nemotron长上下文摘录说明CP64、TP2、EP64，用于极长序列阶段；CP分摊attention上下文，EP分摊专家，它们未必是两个独立额外设备轴。原页总结中的PP=0通常表示未采用/未报告PP，而不能作为真实组大小带入乘法；没有流水线切分时计算上用p=1。仅凭三个数字无法确定总设备数或DP，应保留未知。

编者检查问题

面对类似配置，先问“设备/芯片/节点哪个单位”“这是框架推荐还是实际训练”“是哪一阶段”“稠密层和专家层各自怎样划分”。满足这些条件之后，才能用乘积校验。若资料里缺DP或PP，诚实标未知比凑一个总卡数更可靠。

本节参考（完整书目见讲义末尾）：[slides] · [moe]

Qwen 3

225B-A22B and 30B-A3B

Parallelism Configuration

Model	Mode	TP	PP	EP	Total GPUs	Use Case
Qwen3-30B-A3B	Pretrain	1	1	8	8	Pre-training (single node)
Qwen3-30B-A3B	Full SFT	1	1	8	8	Full supervised finetuning
Qwen3-30B-A3B	LoRA/DoRA	1	1	8	8	PEFT finetuning (single node)
Qwen3-235B-A22B	Pretrain	2	8	32	512	Pre-training (64 nodes)
Qwen3-235B-A22B	Full SFT	2	8	32	512	Full supervised finetuning (64 nodes)
Qwen3-235B-A22B	LoRA/DoRA	2	8	32	512	PEFT finetuning (64 nodes)

Workload family	Hardware	Typical band	Representative shape
DSV3, large-scale	H100	low-to-mid hundreds TFLOPS/GPU, high-teens MFU	TP2, EP64, PP8, DeepEP
DSV3, large-scale	B200	high-hundreds TFLOPS/GPU, mid-teens MFU	TP1, EP32, PP8, DeepEP
DSV3, large-scale	GB200	around 1K TFLOPS/GPU, low-20s MFU	TP1, EP64, PP4, HybridEP
DSV3, large-scale	GB300	above the GB200 band, often mid-20s MFU	TP1, EP64, PP4, HybridEP
Qwen3 235B	H100	low-300s TFLOPS/GPU, around 30% MFU	TP2, EP32, PP8, DeepEP
Qwen3 235B	GB200	high-hundreds TFLOPS/GPU in tuned runs	TP1 or TP2, EP32-64, PP4, HybridEP
Qwen3 30B	H100	low-200s TFLOPS/GPU	TP1, EP8, PP1, DeepEP
Qwen3-Next 80B	GB200	low-300s TFLOPS/GPU in BF16-class runs	TP1, EP32, PP2, HybridEP

Primarily expert parallel up to 8 GPUs, 2/8/32 parallelism for larger models

Overview table of model parallelism

	DP	TP/SP	EP	PP	CP
Deepseek	?? (Zero1)	1	8	16	??
Deepseek v3	?? (Zero1)	1	64	16	??
Yi	?? (Zero1)	>0	1	>0	??
Llama3 405B	128	8	0	16	1
Gemma 2	768	8	0	0	0
Mixtral 8x22 (megatron)	2	4	8	4	1
Nemotron 3 120B (long context)	??	2	64	??	64
Qwen 3 (megatron)	??	2	32	8	1

Patterns – TP generally <= 8. EP can be big (but hard!). Long context phases use large CP

第 8 讲 · 原页 71–72 · 中文讲解

Qwen与总览表：将案例当模式，而非固定配方

第71页的名字与配置

页上写225B-A22B，但嵌入表格写Qwen3-235B-A22B；此处按表格辨认并保留文字差异。30B-A3B与235B-A22B的A表示每token激活参数量，不能据此按激活参数量计算全部模型状态显存。截图区分pretrain、full SFT和LoRA等用途；这些推荐配置不能直接证明官方原始预训练使用完全相同布局。

第72页的零与问号

EP=0通常表示稠密模型不用专家并行，PP/CP=0表示未用的写法；做代数记账时不启用的轴度数是1。问号则是资料未确定，不能补成1。TP/SP合栏还可能掩盖不同激活布局；DeepSeek早期与V3也不能混成同一配置。把第64页文字与总表并读，可看到版本和简写口径的差异。

应带走的三条趋势

TP常限在高速域，EP可能因专家容量跨更大域，长上下文阶段可能提升CP并降低DP。这些都是有硬件和工作负载前提的模式。真正可迁移的是分析顺序：先算模型与激活账本，再看哪个通信量依赖参数、哪个依赖激活，最后匹配可承受的同步频率与网络拓扑。

本节参考（完整书目见讲义末尾）：[slides] · [moe]

Recap for the whole lecture

- Scaling beyond a certain point requires multi-gpu, multi-node parallelism
- No single solution to the parallelism problem (probably want all 3 approaches)
- Simple, interpretable rules of thumb for combining different forms of parallelism

第 8 讲 · 原页 73–73 · 中文讲解

整讲贯通：给一份并行方案做验收

四个必须回答的问题

第一，哪些样本、权重、梯度、优化器状态、激活由本卡负责，哪些复制？第二，前向/反向/更新分别调用什么通信，属于哪个组？第三，容量预算是常驻还是峰值，是否包含微批缓冲、预取和精度转换？第四，速度公式是否声明消息大小、延迟、带宽、批量、负载均衡与重叠假设？答清这四点，就能解释一种布局为什么适合该训练，而不只是报出DP/TP/PP数字。

从学习到实践的顺序

先用两卡向量和小矩阵检查语义，再用本讲状态公式与调度时间线做预算，最后在实际硬件上测量；测量应包含峰值显存、每卡/全群吞吐、通信等待和恢复成本。本讲的算例只做CPU算术验证，不代表实测多GPU性能。下一步理解缩放规律时，要把“增加可用计算预算”与“系统实际能交付多少有效计算”联系起来。

本节参考（完整书目见讲义末尾）：[slides]

自测与参考答案

1. AllGather和AllReduce都让每卡拿到完整向量，为什么不能互换？

AllGather拼接不同片段，AllReduce对相同位置归约。两卡各有[1,2]和[10,20]时，AllGather得[1,2,10,20]，sum-AllReduce得[11,22]。输入/输出长度和数学语义都不同；RS后AG才可复现完整归约。

2. 8卡、8B参数、2字节通信，环AllReduce每卡发多少？基本FSDP呢？

S=16 GB，RS与AG各发7S/8=14 GB；AllReduce发送28 GB，前向/反向各一次AG再一次RS的FSDP发送42 GB。接收量相同，但本文不把它计入发送量；全群发送再乘8。计算忽略启动延迟、拥塞与额外重算/累积。

3. 局部平均梯度为2与8，两卡分别含1与3个有效样本，直接平均得到5是否正确？

若每样本权重相同，全局平均应为(1×2+3×8)/4=6.5。局部batch不等长时不能等权平均各rank均值；语言模型中的有效token数差异同样需要处理。

4. ZeRO-2是否每张卡只反传四分之一模型（d=4）？

不是。每卡仍为本地样本经过整个模型，只把归约后的持久梯度和优化器状态分成四份。层/桶梯度在反向中逐步产生、归约、释放。只计算部分层是PP等模型并行的分工。

5. 8卡、FP16/BF16参数和梯度、FP32主权重与矩，每卡模型状态如何计？

K=12，DDP为16P，ZeRO-1为5.5P，ZeRO-2为3.75P，ZeRO-3为2P字节。这不是第28页12字节设定；那里两矩也用BF16，K=8。两种精度预算不能混用。

6. FSDP按层执行，模型有L层，为什么主导通信不是3LP？

P已是全模型参数总数，若层参数量P_l，则$\Sigma_l 3P_l=3P$。集合操作次数随L增长，延迟项可能增长；载荷本身不再乘L。3P还需前向后释放并在反向重新AG等前提。

7. p=4，m=4时原稿75%的气泡比是否意味着利用率25%？

不是。75%是气泡/有用工作；总时间对应4+3=7个等效单位，利用率4/7≈57.1%，气泡占总时间3/7≈42.9%。只有单微批m=1时，理想利用率才是1/p=25%。

8. TP前向的局部结果为何在非线性之前未必能直接相加？

非线性一般不满足φ(u+v)=φ(u)+φ(v)。列切A把不同输出坐标交给不同卡，各自应用GeLU是合法的；后续行切B产生的是同一输出的可相加线性贡献。错误地提前对求和项独立应用非线性会改变模型。

9. 激活长度翻倍，sbh(34+5as/h)变多少？

其他量固定时，34sbh翻倍，5abs^2变四倍，所以总体不是总能说翻倍或四倍。TP+SP和选择性重算去掉保存的二次项之后，该账本变为34sbh/t，才对s线性；总attention计算仍可能为二次。

10. 64卡、TP8、PP4，DP多少？b=1、m=16时全局batch多少？

在三个轴独立且无其他设备轴时，d=64/(8×4)=2，全局样本数B=2×1×16=32。TP和PP协作同样的样本，不再把它们乘入B。

11. 8卡稠密DP=8，专家EP=4、EDP=2，为什么不是32卡？

专家并行复用了稠密DP的设备，两个4卡EP组共8卡。两份专家集合之间才是EDP同步；切分维度的物理集合不同，不能再把DP8与EP4连乘。

12. Llama常规行改成长上下文行，为什么CP16而DP8仍为16384卡？

TP8与PP16不变，原CP1×DP128=128，改为CP16×DP8仍为128。因此总数同为16384，但每个长序列使用更多协作设备，全局数据副本变少。

13. 重算使同batch一步更慢，为什么还能选它？

若不重算只能运行很小局部batch，而重算释放显存后可以扩大局部batch、降低气泡或改善GEMM，最终可行配置的tokens/sec可能更高。必须在目标优化batch约束下比较，不能说重算无条件加速。

参考资料

[slides] CS336 Spring 2026 Lecture 8 原课件；本稿的页序与主要内容基准

https://github.com/stanford-cs336/lectures/blob/main/lecture_08.pdf

[course] CS336官方课程主页；核对**Spring 2026**及第**8**讲对应关系

https://cs336.stanford.edu/

[captions] 第8讲公开录像字幕的第三方缓存；未由教师审校，用于时间定位

https://jingzechen.github.io/assets/courses/stanford-cs336-s26/lectures/008/captions.source.srt

[nccl] NVIDIA NCCL Collective Operations；官方集合操作语义

https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/usage/collectives.html

[zero] Rajbhandari等：ZeRO；原论文，状态分片与通信分析

https://arxiv.org/html/1910.02054v3

[fsdp] Zhao等：PyTorch FSDP；原论文，单元分片与通信计算调度

https://arxiv.org/html/2304.11277v1

[fsdp-doc] PyTorch官方FSDP教程；按需参数汇集与梯度分片

https://docs.pytorch.org/tutorials/intermediate/FSDP_tutorial.html

[megatron] Narayanan等：Efficient Large-Scale Language Model Training；原论文，**TP/PP/DP与实测案例**

https://arxiv.org/abs/2104.04473

[activation] Korthikanti等：Reducing Activation Recomputation；原论文，特定**Transformer激活账本**

https://arxiv.org/abs/2205.05198

[zero-bubble] Qi等：Zero Bubble Pipeline Parallelism；原论文，拆分反向与调度边界

https://arxiv.org/abs/2401.10241

[ring] Liu等：Ring Attention with Blockwise Transformers；原论文，长上下文分块通信

https://arxiv.org/abs/2310.01889

[moe] NVIDIA Megatron Core官方MoE文档；并行组折叠与配置建议，内容可能随版本更新

https://docs.nvidia.com/megatron-core/developer-guide/latest/user-guide/features/moe.html

[scaling-book] JAX How To Scale Your Model：训练并行分析；原页**56**所引技术说明

https://jax-ml.github.io/scaling-book/training/

[deepseek] DeepSeek-V3 Technical Report；原报告，**PP/EP与DualPipe**

https://arxiv.org/html/2412.19437v2

[llama] The Llama 3 Herd of Models；原报告，分阶段布局与故障统计

https://arxiv.org/abs/2407.21783